

Article

Grounded Knowledge Graph Extraction via LLMs: An Anchor-Constrained Framework with Provenance Tracking

Yuzhao Yang ^{1,2}, Genlang Chen ^{1,*}, Binhua He ³ and Yan Zhao ^{1,*} 

¹ School of Computer and Data Engineering, Ningbo Tech University, Ningbo 315199, China; yuzhao.yang@nit.zju.edu.cn

² School of Computer Science and Engineering, Southeast University, Nanjing 211189, China

³ School of Computer Science and Technology (School of Artificial Intelligence), Zhejiang Sci-Tech University, Hangzhou 310018, China; 2023220603022@mails.zstu.edu.cn

* Correspondence: cgl@zju.edu.cn (G.C.); zhaoyan@nit.zju.edu.cn (Y.Z.)

Abstract

Knowledge graphs represent real-world facts as structured triplets and underpin a wide range of applications, including question answering, recommendation, and retrieval-augmented generation. Automatically extracting such triplets from unstructured text is essential for scalable knowledge base construction. Traditional extraction methods require task-specific training data and struggle to generalize across domains. Large language models (LLMs) offer an alternative through in-context learning, enabling flexible extraction without fine-tuning. However, LLMs frequently hallucinate—generating plausible triplets unsupported by the source text. The root cause is the lack of provenance: existing methods produce triplets without explicit links to their textual origins, making faithfulness unverifiable. This paper presents Anchor-Extraction-Verification-Supplement (AEVS), a framework that grounds every triplet element to the source text. AEVS operates in three stages: (1) anchor discovery identifies entities, relation phrases, and attribute values with precise positions, forming a constrained extraction vocabulary; (2) grounded extraction generates triplets linked to discovered anchors; and (3) restoration-based verification validates triplets through hierarchical matching, with a coverage-aware supplement ensuring comprehensive extraction. Experiments on WebNLG, REBEL, and Wiki-NRE demonstrate consistent improvements over both trained models and LLM-based baselines. Ablation studies confirm that anchor-based constraints are the primary mechanism for hallucination reduction. Dedicated analyses of anchor discovery quality, computational cost (2.83–4.28 LLM calls per sample), and hallucination rates (0.23–20.23% across model–dataset configurations) provide insights into the framework’s practical applicability and limitations.

Keywords: knowledge graph extraction; large language models; text grounding; provenance tracking



Academic Editor: Paolo Bellavista

Received: 13 January 2026

Revised: 7 February 2026

Accepted: 3 March 2026

Published: 9 March 2026

Copyright: © 2026 by the authors.

Licensee MDPI, Basel, Switzerland.

This article is an open access article distributed under the terms and conditions of the [Creative Commons Attribution \(CC BY\)](https://creativecommons.org/licenses/by/4.0/) license.

1. Introduction

Knowledge graphs organize real-world facts as structured (subject, relation, object) triplets, serving as critical infrastructure for a wide range of intelligent applications, including question answering, recommendation systems, semantic search, and retrieval-augmented generation [1,2]. Knowledge graph extraction (KGE)—automatically transforming unstructured text into such triplets—is essential for building and maintaining large-scale knowledge bases. Given the exponential growth of textual data across the web,

the scientific literature, and enterprise documents, manual curation is infeasible, making automated extraction indispensable. The quality of extracted knowledge directly impacts all downstream applications; erroneous triplets propagate through reasoning chains, contaminate retrieval results, and ultimately undermine user trust in knowledge-driven systems.

Traditional KGE approaches have evolved through several paradigms, each with inherent limitations. Pipeline methods decompose extraction into sequential stages; named entity recognition (NER) first identifies entity mentions, followed by relation classification (RC), which determines the relationships between entity pairs [3]. While conceptually straightforward, these methods suffer from error propagation, where mistakes in entity recognition cascade through subsequent stages and an unrecognized entity cannot participate in any downstream relation. Joint extraction methods emerged to address this limitation by simultaneously modeling entities and relations within a unified framework [4–6]. Although achieving improved performance, these methods require substantial task-specific training data and depend on predefined relation schemas, limiting their ability to generalize across domains. Generative approaches reframe KGE as sequence-to-sequence generation [7,8] but still require extensive fine-tuning on domain-specific data. Crucially, none of these methods provide mechanisms to verify whether generated triplets are actually grounded in the source text.

Large Language Models (LLMs) offer a promising alternative through in-context learning, enabling extraction via carefully designed prompts without task-specific training [9–11]. This paradigm shift promises unprecedented flexibility: LLMs can potentially handle arbitrary relation types and adapt to new domains through prompt engineering alone. However, LLM-based approaches introduce a critical challenge of hallucination. LLMs frequently generate triplets that appear plausible but are not actually supported by the source text [12]. For example, given the sentence “*Steve Jobs co-founded Apple Inc. in Cupertino in 1976*”, an LLM might correctly extract (Steve Jobs, founder, Apple Inc.) and (Apple Inc., foundingYear, 1976), but also hallucinate (Steve Jobs, nationality, American) or (Steve Jobs, education, Reed College)—facts that may be true in the real world but are *not stated* in the given text. Such hallucinations pose severe risks: they are difficult to detect automatically since the model provides no evidence for its claims; they may introduce misinformation when the hallucinated fact happens to be false; and they fundamentally undermine trust in the entire knowledge graph, as users cannot distinguish verified extractions from fabricated ones.

The root cause of hallucination lies in the *ungrounded* nature of existing extraction approaches. Whether using traditional models or LLMs, current methods produce triplets as standalone outputs without explicit connections between triplet elements and their source positions in the original text. This lack of *provenance*—the ability to trace each piece of extracted information back to its textual origin—makes it fundamentally impossible to verify extraction faithfulness. The model may draw upon its parametric knowledge, linguistic patterns, or statistical associations rather than the actual content of the input text.

In this paper, we propose anchor-extraction-verification-supplement (AEVS), a novel framework that addresses the hallucination problem by ensuring complete traceability in LLM-based knowledge graph extraction. Our key insight is both simple and powerful: *if every element of a triplet can be traced back to a specific span in the source text, then hallucinations become detectable and eliminable*. AEVS operationalizes this insight through three carefully designed stages:

- (1) **Anchor Discovery.** Rather than directly extracting triplets, we first systematically identify all meaningful information “anchors” in the text—entities, relation phrases, and attribute values—along with their precise character-level positions. This creates a comprehensive inventory of *what information actually exists* in the text. Relation anchors

are additionally mapped to canonical schema relations where applicable, enabling normalization across diverse surface expressions (e.g., “founded”, “co-founded”, and “established” all map to founder). The anchor set establishes a closed vocabulary of elements that can legitimately appear in triplets; any triplet element not traceable to an anchor immediately signals a potential hallucination.

- (2) **Grounded Extraction.** Triplets are extracted with explicit constraints requiring each element to be linked to discovered anchors. Each triplet element (subject, relation, object) receives a binary grounding status indicating whether it was successfully linked to source evidence. This architectural constraint fundamentally limits the model’s ability to fabricate information beyond what the text contains.
- (3) **Restoration Verification with Coverage-Aware Supplement.** We verify each triplet through a novel “extract-then-restore” paradigm; for each element, we attempt to restore it back to the original anchors using a hierarchy of matching strategies: exact matching, fuzzy matching for minor variations, schema-based matching for relation normalization, and text search as a fallback. The element-level restoration status determines the verification path; *fully restored* triplets (all three elements successfully matched) are directly accepted with high confidence, while *partially restored* triplets undergo targeted LLM verification, focusing on ungrounded elements. To ensure comprehensive extraction, we further monitor which anchors remain unused after initial extraction. When significant anchors lack corresponding triplets, the system triggers additional extraction rounds specifically targeting these unused anchors. Semantic deduplication using sentence embeddings prevents redundant triplets across extraction rounds, maintaining precision while improving recall.

Our contributions are summarized as follows:

- We propose AEVS, a three-stage framework that ensures complete traceability in LLM-based knowledge graph extraction, providing character-level provenance for every triplet element and enabling principled hallucination detection.
- We design a *hierarchical restoration* mechanism with four complementary matching strategies (exact, fuzzy, schema-based, and text search) that effectively distinguishes faithful extractions from hallucinations based on element-level restoration status.
- We introduce a *coverage-aware supplement* mechanism with semantic deduplication that monitors anchor utilization and extracts additional triplets for unused anchors, ensuring comprehensive extraction while maintaining precision.
- Extensive experiments on three benchmarks (WebNLG, REBEL, and Wiki-NRE) demonstrate that AEVS significantly outperforms both specialized trained models and existing LLM-based approaches, achieving up to a 0.412 absolute improvement in F1 score. We further provide dedicated analyses of anchor discovery quality, computational cost, hallucination rates, and qualitative error patterns to characterize the framework’s practical applicability and limitations.

We note that the current framework operates at the sentence level with access to a predefined relation schema, focusing on grounding triplet elements to explicit textual evidence. Extensions to cross-sentence extraction, open-domain settings without predefined schemas, and multilingual corpora are discussed as future work in Section 5.

2. Related Work

Knowledge graph extraction (KGE) aims to automatically extract structured triplets (subject, relation, object) from unstructured text. In this section, we provide a comprehensive review of KGE methods, covering traditional approaches, generative methods, and recent LLM-based techniques.

2.1. Traditional Knowledge Graph Extraction

Traditional KGE approaches can be categorized into pipeline-based and joint extraction methods. Pipeline approaches decompose the task into named entity recognition followed by relation classification [3,13,14]. While conceptually simple, these methods suffer from error propagation, where mistakes in entity recognition cascade to relation extraction. Pretrained language models improved pipeline methods [15,16], but the fundamental limitation remains.

Joint extraction methods address error propagation by simultaneously identifying entities and relations. Various modeling paradigms have been explored. Sequence labeling approaches assign composite tags encoding both entity and relation information [4]; span-based methods enumerate candidate spans and classify entity-relation pairs [17–19]; table filling methods represent extraction as populating a token-pair matrix [20,21]; and cascade decomposition methods first identify head entities and then extract corresponding relations and tail entities [5,6]. Graph neural network-based approaches model sentences as graphs to capture long-range dependencies [22–24]. These methods achieve strong performance but require task-specific training data and predefined schemas.

2.2. Generative Information Extraction

Recent work reframes information extraction as sequence generation, leveraging pretrained seq2seq models. Early approaches used copy mechanisms to generate entities from source sentences [25,26]. REBEL [7] proposed a linearization scheme that expresses triplets as text sequences, achieving state-of-the-art performance with BART pretraining on over 200 relation types. ReGen [27] employed reinforcement learning to directly optimize evaluation metrics, establishing strong results on WebNLG+ 2020. GenIE [8] introduced constrained generation by using prefix tries to ensure the extracted triplets conform to predefined schemas, enabling scaling to large knowledge bases. These generative methods eliminate pipeline decomposition but do not provide provenance for extracted triplets.

2.3. LLM-Based Knowledge Graph Construction

Large language models enable KGE through prompting without task-specific training. Early work like ChatIE [9] decomposed extraction into multi-turn dialogues, while comprehensive evaluations [28,29] revealed that LLMs perform better as reasoning assistants than few-shot extractors, with significant performance degradation on complex triplet structures. GPT-RE [10] addressed demonstration relevance through task-aware retrieval and gold label-induced reasoning chains, achieving competitive performance on relation extraction benchmarks.

EDC [11] proposed a three-phase framework to handle large schemas that exceed context limits: (1) open information extraction without schema constraints, (2) generating natural language definitions for discovered relations, and (3) post hoc canonicalization to standardize triplets. This approach enables flexible extraction but cannot recover from errors in the initial extraction phase. Other approaches include CodeKGC [30], which frames extraction as code generation, instruction tuning for zero-shot extraction [31], and GraphRAG [2] for schema-free construction.

2.4. Summary

Existing methods face a common limitation: they lack mechanisms to trace extracted triplets back to specific source text positions. Our AEVS framework addresses this through anchor-grounded extraction with character-level provenance. Unlike EDC's post hoc canonicalization, AEVS identifies text anchors first and grounds extraction from the start. The restoration-based verification enables automatic quality assessment by independently matching triplets back to source positions.

3. Method

3.1. Problem Formulation

Let $\mathcal{T} = \langle c_1, c_2, \dots, c_n \rangle$ denote an input text comprising n characters. The objective of *knowledge graph extraction* (KGE) is to derive a knowledge graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ from $\mathcal{T}$, represented as a set of relational triplets $\mathcal{K} = \{\tau_i = (s_i, r_i, o_i)\}_{i=1}^m$, where each triplet τ consists of a subject entity $s \in \mathcal{V}$, a relation $r \in \mathcal{E}$, and an object $o \in \mathcal{V}$.

Beyond semantic correctness, we impose a **traceability constraint**: each triplet element must be *provenance-linked* to a specific character span $[p^{\text{start}}, p^{\text{end}})$ in the source text $\mathcal{T}$. Formally, we define the provenance function $\pi : \mathcal{K} \times \{s, r, o\} \rightarrow 2^{[0, n]}$ that maps each triplet element to the set of character positions in $\mathcal{T}$ that constitute its supporting evidence. Concretely, $\pi(\tau, e)$ returns the character span $[p^{\text{start}}, p^{\text{end}})$ from which element e of triplet τ was derived; the power set notation $2^{[0, n]}$ accommodates elements that may be supported by multiple or zero spans. A triplet τ is *faithful* if and only if $\pi(\tau, e) \neq \emptyset$ for all $e \in \{s, r, o\}$.

3.2. Framework Overview

AEVS addresses the hallucination problem through a principled *anchor-constrained extraction* paradigm. The core insight is that hallucinations arise from the unconstrained generation space of LLMs; by establishing a closed vocabulary of text-grounded elements *before* extraction, we fundamentally limit the model's ability to fabricate information.

As illustrated in Figure 1, AEVS operates through three sequential stages with well-defined interfaces:

Stage 1: Anchor Discovery ($\mathcal{T} \rightarrow \mathcal{A}$). Given the input text $\mathcal{T}$, we identify all semantically meaningful spans—entities, relation phrases, and attribute values—along with their precise character positions. The output is a comprehensive anchor set $\mathcal{A} = \mathcal{A}_E \cup \mathcal{A}_R \cup \mathcal{A}_T$ that establishes the vocabulary of elements that can legitimately appear in triplets.

Stage 2: Grounded Extraction ($(\mathcal{T}, \mathcal{A}) \rightarrow \mathcal{K}_{\text{raw}}$). Triplets are extracted under explicit grounding constraints; each element must be linked to a discovered anchor. This transforms KGE from an open-ended generation problem to a constrained combinatorial selection problem, where the LLM selects and combines anchors rather than freely generating text.

Stage 3: Restoration-Based Verification and Coverage-Aware Supplement ($(\mathcal{K}_{\text{raw}}, \mathcal{A}, \mathcal{T}) \rightarrow \mathcal{K}^*$). Each triplet undergoes independent verification through a novel “extract-then-restore” paradigm. We attempt to restore each element back to its source anchor using hierarchical matching strategies. The restoration status determines the verification path; fully restored triplets are directly accepted, while partially restored ones undergo targeted LLM verification. A coverage-aware supplement mechanism then extracts additional triplets for unused anchors, ensuring comprehensive coverage.

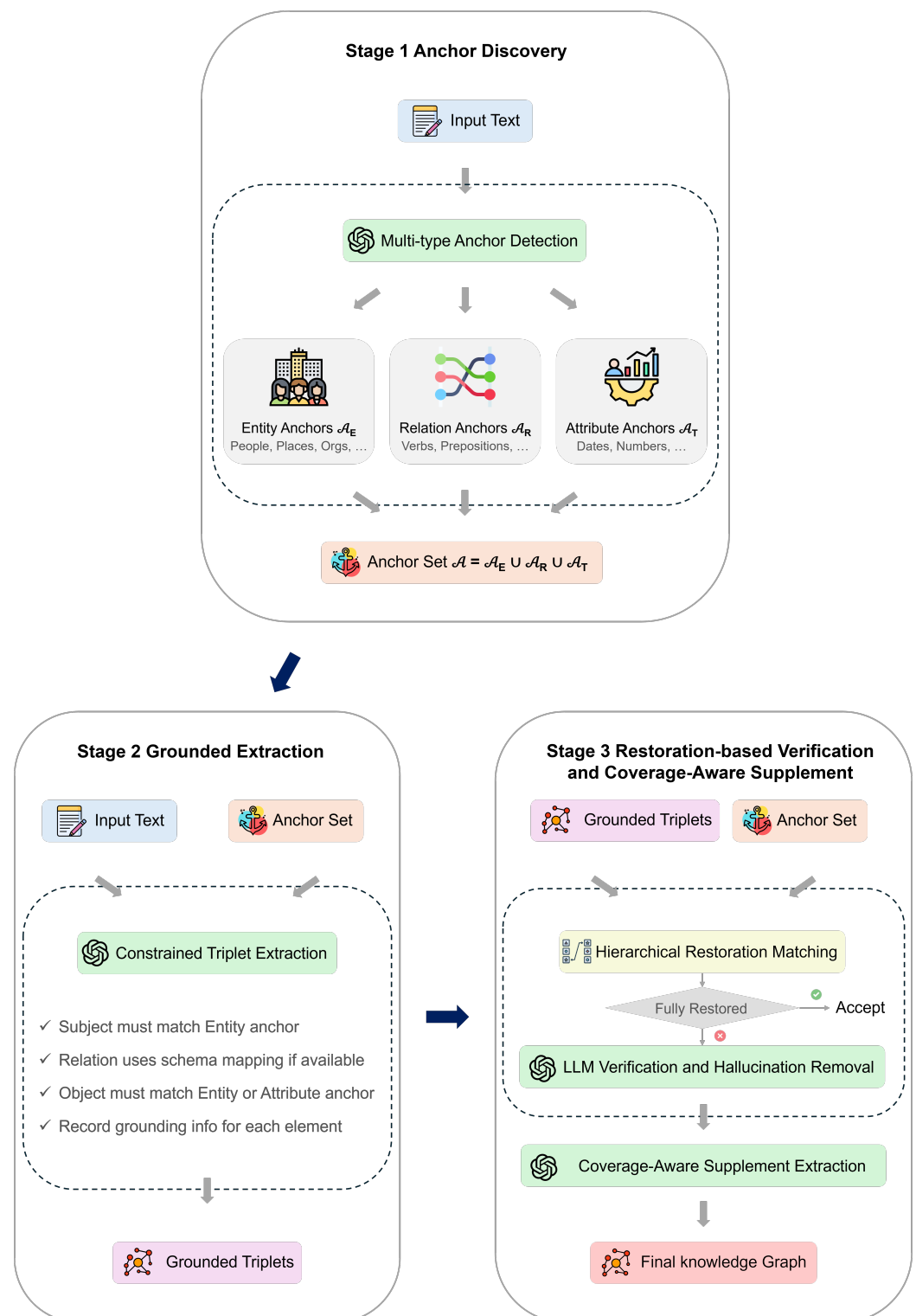


Figure 1. The AEVS framework architecture. Blue labels denote inputs, green labels denote LLM processing steps. Arrows indicate data flow between stages. **Stage 1 (anchor discovery)** identifies entity, relation, and attribute anchors with character-level positions, forming a closed vocabulary $\mathcal{A}$. **Stage 2 (grounded extraction)** extracts triplets constrained by $\mathcal{A}$, producing grounded triplets $\mathcal{K}_{\text{raw}}$ with provenance annotations. **Stage 3 (restoration-based verification and coverage-aware supplement)** verifies each triplet through independent restoration matching; fully restored triplets are accepted directly, while partially restored ones undergo LLM verification. A coverage-aware supplement mechanism ensures comprehensive extraction by targeting unused anchors.

3.3. Scope and Assumptions

Before detailing the pipeline stages, we make explicit several assumptions and scope limitations that inform the framework design and its current applicability.

AEVS assumes access to a predefined relation schema $\mathcal{R}$ for schema mapping during anchor discovery (Stage 1) and schema-based restoration matching (Stage 3). When no schema is available, the framework can operate in a *schema-free mode* by disabling the schema mapping function ($\varphi \equiv \emptyset$) and relying solely on exact, fuzzy, and text search matching during restoration. However, this mode reduces relation normalization quality and may increase the rate of unsupported-relation errors, as surface-form diversity among LLM-generated relation phrases will not be canonicalized. The impact of schema completeness on performance is partially visible in the dataset-specific results; REBEL (200 relation types) exhibited higher relation miss rates than Wiki-NRE (45 types), suggesting that larger, more comprehensive schemas facilitate better extraction (see Section 4.5).

The current implementation processes each input text independently at the sentence or passage level, without explicit handling of cross-sentence coreference or document-level relations. Relations spanning multiple sentences (e.g., a pronoun referring to an entity introduced earlier) would require an additional coreference resolution step before anchor discovery. Extending anchor tracking across sentence boundaries is a natural future direction (Section 5).

The character-level position tracking central to AEVS assumes consistent UTF-8 encoding, which covers the majority of practical scenarios including English and most European languages. Sensitivity to alternative encodings, CJK tokenization boundaries, or mixed-script texts has not been systematically evaluated and represents a limitation for multilingual deployment.

3.4. Stage 1: Anchor Discovery

3.4.1. Motivation

Existing LLM-based extraction methods directly generate triplets from text without constraint, providing the model with unlimited freedom to fabricate plausible-sounding facts not present in the source. Our key insight is to establish an *information inventory* before extraction. By systematically identifying all meaningful spans in the text along with their positions, we create a closed set $\mathcal{A}$ of legitimate triplet elements. This serves three purposes: (1) constraining the extraction space to text-grounded elements; (2) enabling provenance tracking for verification; and (3) supporting coverage monitoring for comprehensive extraction.

3.4.2. Anchor Type Taxonomy

We formalize anchors as structured tuples encoding text content, semantic type, positional information, and type-specific metadata. Let Σ^* denote the set of all strings over alphabet Σ , and let $\mathbb{N}$ denote the set of natural numbers.

Definition 1 (Anchor). An anchor is a five-tuple $a = (\text{text}, \text{type}, p^{\text{start}}, p^{\text{end}}, \mu)$ where the following apply:

- $\text{Text} \in \Sigma^*$ is the surface text span extracted from the source;
- $\text{Type} \in \{\text{entity}, \text{relation}, \text{attribute}\}$ is the semantic category;
- $(p^{\text{start}}, p^{\text{end}}) \in \mathbb{N}^2$ defines the character-level span satisfying $0 \leq p^{\text{start}} < p^{\text{end}} \leq |\mathcal{T}|$;
- μ is type-specific metadata. For entity anchors, μ specifies the entity subtype (e.g., *person*, *organization*); for relation anchors, μ stores the schema mapping result (e.g., “co-founded” $\rightarrow$ *founder*); and for attribute anchors, μ specifies the value type (e.g., *year*, *date*).

Based on their semantic roles in triplet construction, we define three specialized anchor categories.

Definition 2 (Entity anchor). *An entity anchor represents a named entity that can serve as the subject or object in a triplet. The entity anchor set is defined as*

$$\mathcal{A}_E = \{a : a.type = \textit{entity}, a.\mu \in \Omega_E\} \quad (1)$$

where $\Omega_E = \{\textit{person}, \textit{location}, \textit{organization}, \textit{product}, \textit{event}, \textit{work}, \dots\}$ is the entity type ontology and the metadata μ specifies the fine-grained entity subtype.

Definition 3 (Relation anchor). *A relation anchor captures a phrase expressing a relationship between entities. The relation anchor set is defined as*

$$\mathcal{A}_R = \{a : a.type = \textit{relation}, a.\mu = \phi(a.text)\} \quad (2)$$

where $\phi : \Sigma^* \rightarrow \mathcal{R} \cup \{\emptyset\}$ is a schema mapping function that normalizes surface phrases to a predefined relation schema $\mathcal{R}$ or returns $\emptyset$ if no canonical mapping applies. For example, $\phi(\textit{"co-founded"}) = \textit{founder}$, and $\phi(\textit{"was born in"}) = \textit{birthPlace}$.

The schema mapping ϕ serves two critical purposes: (1) it enables relation normalization across diverse surface expressions (e.g., “founded”, “co-founded”, and “established” all map to `founder`), and (2) it facilitates schema-based restoration matching in Stage 3.

Definition 4 (Attribute anchor). *An attribute anchor represents a literal value that describes an entity property, typically serving as the object in attribute triplets. The attribute anchor set is defined as*

$$\mathcal{A}_T = \{a : a.type = \textit{attribute}, a.\mu \in \Omega_T\} \quad (3)$$

where $\Omega_T = \{\textit{date}, \textit{year}, \textit{number}, \textit{quantity}, \textit{title}, \textit{duration}, \textit{percentage}, \dots\}$ is the attribute type ontology. Unlike entity anchors, attribute anchors denote literal values rather than references to real-world entities.

Definition 5 (Complete anchor set). *The complete anchor set $\mathcal{A}$ is the union of all three anchor categories:*

$$\mathcal{A} = \mathcal{A}_E \cup \mathcal{A}_R \cup \mathcal{A}_T \quad (4)$$

which establishes the closed vocabulary of elements that can legitimately appear in extracted triplets.

3.4.3. Discovery Process

The anchor discovery process employs an LLM $\mathcal{M}$ with a structured prompt template (Box 1) that provides explicit type definitions, emphasizes positional precision, and includes schema mapping guidance.

The prompt template incorporates several design principles to maximize anchor discovery quality. The *[Original Text]* section provides the source document, while *[Available Schema Relations]* supplies the predefined relation vocabulary to guide schema mapping. The *Anchor Types* section defines the three categories with concrete examples, helping the LLM distinguish between entities, relations, and attributes. The *Rules* section emphasizes two critical requirements: (1) completeness of relation phrases (avoiding fragmented spans like splitting “was born in” into “was” and “born in”) and (2) precise character-level positions, which are essential for provenance tracking. The *Schema Mapping Examples* demonstrate how diverse surface expressions should be normalized to canonical relations,

enabling consistent downstream processing. Finally, the structured *JSON output format* facilitates reliable parsing of the LLM response.

Box 1: Anchor Discovery Prompt Template

```

Identify all anchors (meaningful information spans) in the text.

[Original Text]
{text}

[Available Schema Relations]
{schema_relations}

=== Anchor Types ===
- ENTITY: Named entities (people, places, organizations, products...)
- RELATION: Relation phrases (verbs or prepositional phrases expressing
relationships)
- ATTRIBUTE: Attribute values (dates, numbers, titles, quantities...)

=== Rules ===
1. Identify ALL meaningful phrases with appropriate types
2. Relation phrases should be complete (e.g., "was born in" not split)
3. Provide PRECISE character positions (char_start, char_end)
4. For RELATION type: Map to schema relation if possible (CRITICAL)

=== Schema Mapping Examples ===
- "was born in" / "born in" -> birthPlace
- "died in" / "passed away in" -> deathPlace
- "founded" / "co-founded" / "established" -> founder
- "is located in" / "located in" / "is in" -> location
- "is the CEO of" / "leads" / "heads" -> CEO

=== Output Format (JSON) ===
{"anchors": [{"text": "...", "type": "entity|relation|attribute",
"char_start": N, "char_end": M, "schema_mapping": "..."}]}
```

After receiving the LLM response, we apply rigorous validation to ensure anchor quality. Algorithm 1 details the complete discovery and validation procedure.

Algorithm 1 implements a robust anchor discovery process with comprehensive validation. The algorithm first invokes the LLM with the formatted prompt (lines 1–2) and parses the JSON response to obtain raw anchor candidates (line 3). Each candidate then undergoes a three-phase validation pipeline:

Phase 1: Position Bound Validation (lines 6–8). We verify that the reported character positions are within valid ranges; the start position must be non-negative, the end position cannot exceed the text length, and the start must precede the end. Anchors failing these basic sanity checks are discarded.

Phase 2: Text-Position Consistency Verification (lines 10–17). LLMs frequently report incorrect character positions due to tokenization differences or counting errors. We extract the substring at the claimed position and compare it with the anchor’s declared text. If they are mismatched, then we attempt automatic correction by searching for the anchor text in the full document. This self-healing mechanism recovers many anchors that would otherwise be lost due to minor position errors.

Phase 3: Schema Mapping Validation (lines 19–22). For relation anchors with schema mappings, we verify that the mapped relation exists in the predefined schema $\mathcal{R}$. Invalid mappings (e.g., typos like “locaton” instead of “location”) are corrected through fuzzy string matching against the schema.

Algorithm 1 Anchor Discovery with Validation**Require:** Text $\mathcal{T}$, Schema $\mathcal{R}$, LLM $\mathcal{M}$ **Ensure:** Validated anchor set $\mathcal{A}$

```

1: prompt  $\leftarrow$  FORMATANCHORPROMPT( $\mathcal{T}, \mathcal{R}$ )
2: response  $\leftarrow \mathcal{M}(\text{prompt})$ 
3:  $\mathcal{A}_{\text{raw}} \leftarrow \text{PARSEJSON}(\text{response})$ 
4:  $\mathcal{A} \leftarrow \emptyset$ 
5: for  $a \in \mathcal{A}_{\text{raw}}$  do
6:                                      $\triangleright$  Validate position bounds
7:   if  $a.p^{\text{start}} < 0 \vee a.p^{\text{end}} > |\mathcal{T}| \vee a.p^{\text{start}} \geq a.p^{\text{end}}$  then
8:     continue
9:   end if
10:                                      $\triangleright$  Verify text-position consistency
11:   extracted  $\leftarrow \mathcal{T}[a.p^{\text{start}} : a.p^{\text{end}}]$ 
12:   if NORMALIZE(extracted)  $\neq$  NORMALIZE( $a.\text{text}$ ) then
13:      $(p_s, p_e) \leftarrow \text{FINDTEXTPOSITION}(a.\text{text}, \mathcal{T})$ 
14:     if  $(p_s, p_e) \neq \text{None}$  then
15:        $a.p^{\text{start}}, a.p^{\text{end}} \leftarrow p_s, p_e$   $\triangleright$  Auto-correct
16:     else
17:       continue  $\triangleright$  Cannot locate in text
18:     end if
19:   end if
20:                                      $\triangleright$  Validate schema mapping for relations
21:   if  $a.\text{type} = \text{relation} \wedge a.\mu \neq \emptyset$  then
22:     if  $a.\mu \notin \mathcal{R}$  then
23:        $a.\mu \leftarrow \text{FUZZYMATCHSCHEMA}(a.\mu, \mathcal{R})$ 
24:     end if
25:   end if
26:    $\mathcal{A} \leftarrow \mathcal{A} \cup \{a\}$ 
27: end for
28: return  $\mathcal{A}$ 

```

Only anchors passing all three validation phases are included in the final anchor set $\mathcal{A}$. This rigorous validation is essential because subsequent stages depend on anchor accuracy; errors in anchor positions would propagate to incorrect provenance annotations and unreliable verification.

3.5. Stage 2: Grounded Extraction

3.5.1. Grounding Constraints

With the anchor inventory $\mathcal{A}$ established, we extract triplets under explicit grounding constraints. Let $\tau = (s, r, o)$ be a candidate triplet. We define the *grounding function* γ that maps each element to its source anchor:

$$\gamma : \{s, r, o\} \rightarrow \mathcal{A} \cup \{\perp\} \quad (5)$$

where $\gamma(e) = \perp$ indicates that element e could not be grounded to any anchor. The extraction process enforces the following constraints:

$$\gamma(s) \in \mathcal{A}_E \quad (\text{subject must be an entity anchor}) \quad (6)$$

$$\gamma(r) \in \mathcal{A}_R \cup \{\phi^{-1}(r)\} \quad (\text{relation via anchor or schema}) \quad (7)$$

$$\gamma(o) \in \mathcal{A}_E \cup \mathcal{A}_T \quad (\text{object: entity or attribute}) \quad (8)$$

This formulation transforms triplet extraction from an open-ended generation problem to a *constrained combinatorial selection* problem over $\mathcal{A}$. The search space is reduced from $\Sigma^* \times \Sigma^* \times \Sigma^*$ to $\mathcal{A}_E \times (\mathcal{A}_R \cup \mathcal{R}) \times (\mathcal{A}_E \cup \mathcal{A}_T)$, fundamentally limiting hallucination potential.

3.5.2. Extraction with Provenance

The extraction prompt (Box 2) presents discovered anchors with positions and instructs the model to ground each triplet element, recording provenance information.

Box 2: Grounded Extraction Prompt Template

```
Extract knowledge triplets from the text using the identified anchors.
[Original Text]
{text}
[Identified Anchors]
Entity Anchors (use as subject/object):
- 'Steve Jobs' @0-10 [person]
- 'Apple Inc.' @22-32 [organization]
- 'Cupertino' @36-45 [location]
Relation Anchors (use for relations):
- 'co-founded' -> founder @11-21
Attribute Anchors (use as object for attributes):
- '1976' @49-53 [year]
=== EXTRACTION RULES ===
1. CRITICAL: Each triplet element MUST correspond to an anchor
- Subject: Must match an ENTITY anchor
- Relation: Use RELATION anchor's schema_mapping if available
- Object: Must match an ENTITY or ATTRIBUTE anchor
2. GROUNDING: For each element, specify the source anchor
- Include anchor_text and char_start/char_end positions
3. COVERAGE: Try to use ALL anchors in at least one triplet
4. NO HALLUCINATION: Only extract information present in text
=== Output Format (JSON) ===
{"triplets": [{"subject": "...", "relation": "...", "object": "...",
"subject_grounding": {"anchor_text": "...", "char_start": N, "char_end": M},
"relation_grounding": {...}, "object_grounding": {...},
"confidence": 0.95}]}
```

The extraction prompt embodies the core principle of *constrained generation*. Rather than allowing the LLM to freely generate triplet elements, we present the discovered anchors as a closed vocabulary and require explicit grounding for each element. The *[Identified Anchors]* section lists all available anchors organized by type, with their character positions serving as unique identifiers. The *EXTRACTION RULES* enforce the grounding constraints; subjects must come from entity anchors, relations should use schema mappings when available, and objects must match either entity or attribute anchors. The *GROUNDING* requirement forces the model to explicitly specify the source anchor for each element, creating an audit trail that enables subsequent verification. The *COVERAGE* guideline encourages comprehensive extraction by prompting the model to utilize all discovered anchors. Finally, the *NO HALLUCINATION* rule explicitly prohibits fabricating information beyond what the text contains, though we do not rely solely on this instruction; the restoration-based verification in Stage 3 provides the actual enforcement mechanism.

Each extracted triplet τ is augmented with grounding annotations, forming a *grounded triplet*:

$$\hat{\tau} = (\tau, \gamma_s, \gamma_r, \gamma_o) = ((s, r, o), \gamma(s), \gamma(r), \gamma(o)) \quad (9)$$

The grounding annotations $\gamma_s, \gamma_r, \gamma_o$ encode the claimed source anchor for each element, including text content and character positions. Importantly, these annotations are *not blindly trusted*; they serve as hints that will be independently verified through restoration matching in Stage 3. This design acknowledges that LLMs may claim false groundings while still leveraging their annotations to accelerate verification.

3.6. Stage 3: Restoration-Based Verification and Coverage-Aware Supplement

3.6.1. The Extract-Then-Restore Paradigm

Even with grounding constraints, extraction errors persist; the LLM may claim false groundings with incorrect positions or incorrectly infer relationships between anchors. We require a verification mechanism that operates *independently* of the extraction process and does not rely on external knowledge bases.

Our key insight is the extract-then-restore paradigm, where if a triplet is faithful to the source text, then we should be able to independently *restore* each element back to the original anchors through deterministic matching. The restoration process provides cross-validation; it is completely decoupled from extraction and relies solely on the anchor set and original text. Restoration failure for any element signals a potential hallucination.

The verification workflow proceeds as follows. For each extracted triplet, we attempt to restore its subject, relation, and object independently using hierarchical matching strategies. Based on the restoration results, triplets are partitioned into two categories: *fully restored* triplets (all three elements successfully matched), which are directly accepted without further verification, and *partially restored* triplets (one or more elements failed to match), which are sent to LLM-based verification with their restoration status as guidance. After verification and deduplication, we check the anchor coverage, and any unused anchors trigger supplement extraction to capture potentially missed information. Figure 2 illustrates this workflow.

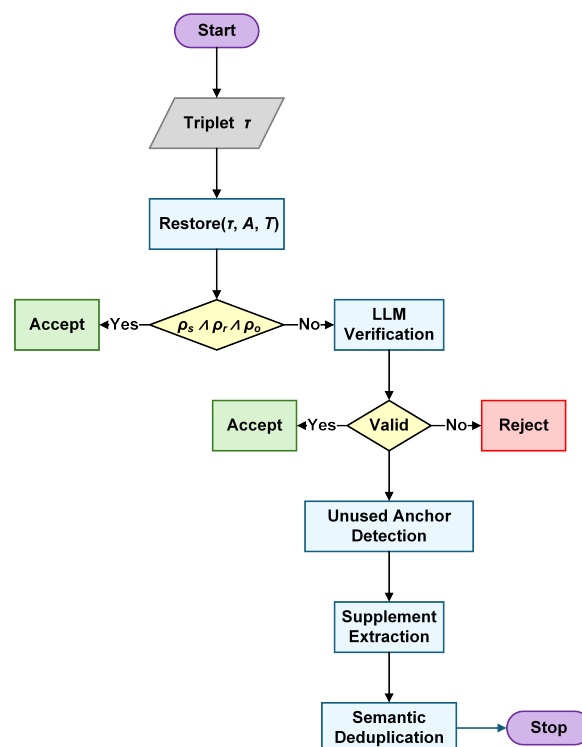


Figure 2. Verification workflow in Stage 3. Each triplet undergoes restoration matching, where fully restored triplets are accepted directly, while partially restored ones are sent to LLM verification. After verification, unused anchors trigger supplement extraction with semantic deduplication.

3.6.2. Hierarchical Restoration Matching

We define a hierarchical restoration function $\mathcal{H}$ that attempts to match each triplet element to anchors or source text through multiple strategies with decreasing confidence. Let e denote a triplet element (subject, relation, or object).

Definition 6 (Restoration Function). *The restoration function $\mathcal{H} : \Sigma^* \times 2^{\mathcal{A}} \times \Sigma^* \rightarrow \{0, 1\} \times (\mathcal{A} \cup \Sigma^*) \times \mathcal{M}$ returns a tuple (ρ, α, m) , where $\rho \in \{0, 1\}$ indicates restoration success, α is the matched anchor or text span, and $m \in \mathcal{M}$ is the matching method.*

The matching method set is $\mathcal{M} = \{m_{\text{exact}}, m_{\text{fuzzy}}, m_{\text{schema}}, m_{\text{text}}\}$, ordered by decreasing confidence. For entity elements (subjects and objects), restoration proceeds through the following hierarchy:

$$\mathcal{H}_E(e, \mathcal{A}, \mathcal{T}) = \begin{cases} (1, a, m_{\text{exact}}) & \exists a \in \mathcal{A}_{E \cup T} : \eta(e) = \eta(a.\text{text}) \\ (1, a, m_{\text{fuzzy}}) & \exists a \in \mathcal{A}_{E \cup T} : \mathcal{F}(e, a) = 1 \\ (1, \mathcal{T}[i:j], m_{\text{text}}) & \exists (i, j) : \eta(e) = \eta(\mathcal{T}[i:j]) \\ (0, \perp, \perp) & \text{otherwise} \end{cases} \quad (10)$$

where $\eta : \Sigma^* \rightarrow \Sigma^*$ is a normalization function (lowercasing and whitespace trimming) and $\mathcal{F}$ is the fuzzy matching predicate, defined by

$$\mathcal{F}(e, a) = \mathbf{1}[\eta(e) \sqsubseteq \eta(a.\text{text}) \vee \eta(a.\text{text}) \sqsubseteq \eta(e)] \quad (11)$$

with $x \sqsubseteq y$ denoting substring containment.

For the relation elements, we additionally incorporate schema-based matching:

$$\mathcal{H}_R(r, \mathcal{A}_R, \mathcal{T}) = \begin{cases} (1, a, m_{\text{schema}}) & \exists a \in \mathcal{A}_R : a.\mu \neq \emptyset \wedge \eta(r) = \eta(a.\mu) \\ (1, a, m_{\text{exact}}) & \exists a \in \mathcal{A}_R : \eta(r) = \eta(a.\text{text}) \\ (1, a, m_{\text{fuzzy}}) & \exists a \in \mathcal{A}_R : \mathcal{F}_R(r, a) = 1 \\ (1, \mathcal{T}[i:j], m_{\text{text}}) & \exists (i, j) : \eta(r) = \eta(\mathcal{T}[i:j]) \\ (0, \perp, \perp) & \text{otherwise} \end{cases} \quad (12)$$

where $\mathcal{F}_R$ extends fuzzy matching with underscore and space normalization:

$$\mathcal{F}_R(r, a) = \mathbf{1}[\zeta(r) \sqsubseteq \zeta(a.\text{text}) \vee \zeta(a.\text{text}) \sqsubseteq \zeta(r)] \quad (13)$$

with $\zeta(x) = \eta(x)$ after removing spaces and underscores.

Table 1 summarizes the matching strategies and their priority order.

Table 1. Hierarchical matching strategies for restoration. Higher-priority methods provide stronger evidence of faithfulness.

Priority	Method	Description	Scope
1	m_{schema}	Match via schema mapping ϕ	Relations only
2	m_{exact}	Exact normalized match with anchor	All elements
3	m_{fuzzy}	Substring containment match	All elements
4	m_{text}	Direct match in source text $\mathcal{T}$	All elements

The matching hierarchy reflects our confidence in different evidence types. Schema mapping (m_{schema}) provides the strongest evidence since it captures semantic equivalence between surface forms and canonical relations (e.g., “co-founded” $\rightarrow$ founder). An exact

anchor match (m_{exact}) indicates that the element directly corresponds to a discovered anchor. Fuzzy matching (m_{fuzzy}) handles minor variations such as abbreviations or partial mentions. Text-based matching (m_{text}) serves as the final fallback, verifying that the element at least appears somewhere in the source text.

For each triplet $\tau = (s, r, o)$, the complete restoration produces a *restoration status vector*:

$$\rho(\tau) = (\rho_s, \rho_r, \rho_o) \in \{0, 1\}^3 \quad (14)$$

where ρ_s, ρ_r, ρ_o are the restoration success indicators for the subject, relation, and object, respectively. We define a triplet as *fully restored* if all elements are successfully matched:

$$\text{FULLYRESTORED}(\tau) \triangleq \rho_s \wedge \rho_r \wedge \rho_o = \mathbf{1}[\|\rho(\tau)\|_1 = 3] \quad (15)$$

3.6.3. Verification Decision Logic

The verification decision is determined by the restoration status vector $\rho(\tau)$. The core principle is simple yet effective; *fully restored triplets are trustworthy, while partially restored ones require additional scrutiny*:

$$\text{DECISION}(\tau, \rho) = \begin{cases} \text{ACCEPT} & \text{if FULLYRESTORED}(\tau) \\ \text{LLMVERIFY}(\tau, \mathcal{T}) & \text{otherwise} \end{cases} \quad (16)$$

This decision logic embodies a key insight: if all three elements of a triplet can be independently traced back to the source text through our hierarchical matching, then the triplet is highly likely to be faithful. Such triplets bypass LLM verification entirely, providing two benefits: (1) *computational efficiency* (fully restored triplets require no additional LLM calls) and (2) *reduced hallucination risk* (the verifier LLM itself could introduce errors, and thus avoiding unnecessary verification improves reliability).

We acknowledge that fully restored triplets are not *guaranteed* to be correct; coincidental string matches could produce false positives where entities co-occur in the text without a genuine relationship between them. However, this risk is mitigated by two factors. First, the anchor-based extraction constrains the entity space to textually grounded mentions, substantially reducing the combinatorial space of spurious entity pairings. Second, our empirical hallucination analysis (Section 4.8) shows that the dominant error mode is unsupported *relations* and not incorrect entity pairings, suggesting that coincidental string matching is not a significant source of false positives in practice.

For partially restored triplets, we employ targeted LLM verification using a carefully designed prompt (Box 3). Crucially, the prompt includes the element-level restoration status, directing the verifier's attention to the specific ungrounded elements. This targeted approach is more effective than asking the LLM to verify the entire triplet without guidance.

The verification prompt is designed with several key features. First, it provides the *original text* as the sole ground truth, ensuring that the verifier bases its judgment only on the source rather than parametric knowledge. Second, it includes the *element-level restoration status* for each triplet, explicitly indicating which elements were found and which were not. This guides the verifier to focus on the problematic elements rather than re-evaluating the entire triplet. Third, the *verification rules* distinguish between explicit statements and valid inferences while rejecting external knowledge, addressing the subtle boundary between acceptable interpretation and hallucination. Finally, the structured *JSON output format* with required reasoning facilitates reliable parsing and enables analysis of verification decisions.

Box 3: LLM Verification Prompt Template

```

Verify if these triplets can be derived from the original text.
[Original Text]
{text}
[Triplets to Verify]
1. (Steve Jobs, deathYear, 2011)
Status: subject 'Steve Jobs' found, relation 'deathYear' NOT found,
object '2011' found in text
2. (Steve Jobs, nationality, American)
Status: subject 'Steve Jobs' found, relation 'nationality' NOT found,
object 'American' NOT found in text
=== Verification Rules ===
A triplet is VALID if:
1. The relationship is explicitly stated in the text
2. OR can be directly inferred from the text (not external knowledge)
3. Even if exact words differ, the MEANING is supported by text
A triplet is INVALID if:
1. The relationship is NOT supported by the text at all
2. Contains hallucinated or fabricated information
3. The subject-object relationship is incorrect or made up
=== Output Format (JSON) ===
{"results": [{"index": 1, "is_valid": true, "reason": "..."}]}

```

After verification, we apply two-tier deduplication to remove redundant triplets. Let $\kappa(\tau) = (\eta(s), \eta(r), \eta(o))$ denote the normalized triplet key. Exact deduplication removes triplets with identical normalized elements:

$$\text{EXACTDUP}(\tau_1, \tau_2) = \mathbf{1}[\kappa(\tau_1) = \kappa(\tau_2)] \quad (17)$$

Fuzzy deduplication further removes substring variants:

$$\text{FUZZYDUP}(\tau_1, \tau_2) = \mathbf{1}[\mathcal{S}(s_1, s_2) \wedge \mathcal{S}(r_1, r_2) \wedge \mathcal{S}(o_1, o_2)] \quad (18)$$

where $\mathcal{S}(x, y) = (x = y) \vee (x \sqsubseteq y) \vee (y \sqsubseteq x)$ is the substring equivalence predicate. At this stage, we use string-based deduplication rather than semantic similarity because the triplets originate from a single extraction pass with consistent relation naming; duplicates typically arise from surface-level variations (e.g., “Apple” vs “Apple Inc.”) rather than semantic paraphrasing.

3.6.4. Coverage-Aware Supplement Extraction

The anchor set $\mathcal{A}$ provides a natural measure of extraction completeness. We define anchor coverage as follows:

$$\text{COVERAGE}(\mathcal{K}, \mathcal{A}) = \frac{|\{a \in \mathcal{A} : \exists \tau \in \mathcal{K}, a \in \text{ANCHORS}(\tau)\}|}{|\mathcal{A}|} \quad (19)$$

where $\text{ANCHORS}(\tau)$ returns the set of anchors referenced by the triplet τ . Unused anchors $\mathcal{A}_{\text{unused}} = \{a \in \mathcal{A} : \nexists \tau \in \mathcal{K}, a \in \text{ANCHORS}(\tau)\}$ indicate potentially missed extractions.

When $|\mathcal{A}_{\text{unused}}| > 0$, we trigger supplement extraction targeting these unused anchors (Box 4).

Box 4: Supplement Extraction Prompt Template

```

Extract ADDITIONAL knowledge triplets using unused anchors.
[Original Text]
{text}
[Already Extracted - DO NOT extract again or semantically similar]
- (Steve Jobs, founder, Apple Inc.)
- (Apple Inc., location, Cupertino)
- (Steve Jobs, position, CEO)
- (Steve Jobs, resignationYear, 2011)
[Unused Anchors - Try to use these]
Attribute: '1976' @49-53 [year]
=== Rules ===
1. Only extract NEW triplets not semantically similar to existing ones
2. Focus on unused anchors
3. Same grounding requirements as before

```

The supplement prompt is carefully designed to maximize recall while preventing redundancy. The *[Already Extracted]* section explicitly lists all triplets that have been verified and accepted, instructing the LLM to avoid extracting duplicates or semantically similar triplets. This negative guidance is crucial; without it, the LLM tends to regenerate the same high-confidence triplets rather than exploring less obvious extractions. The *[Unused Anchors]* section highlights anchors that have not been utilized in any accepted triplet, directing the LLM's attention to potentially missed information. By presenting these unused anchors, we provide concrete targets for the supplement extraction rather than asking the model to blindly search for additional triplets.

Unlike the initial extraction where duplicates arise from surface variations, supplement extraction may produce triplets that are semantically equivalent to existing ones but use different phrasings. Since the LLM generates new triplets in a separate pass, it may express the same fact differently (e.g., “Steve Jobs founded Apple” vs. “Apple was founded by Steve Jobs”). String-based deduplication would miss such cases. Therefore, we apply embedding-based semantic filtering:

$$\text{SEMANTICDUP}(\tau_1, \tau_2; \delta) = \mathbf{1}[\cos(\mathbf{e}_{\tau_1}, \mathbf{e}_{\tau_2}) \geq \delta] \quad (20)$$

where $\mathbf{e}_{\tau} = \text{ENCODE}(\text{concat}(s, r, o))$ is the sentence embedding of the triplet text using a pretrained sentence transformer (e.g., all-MiniLM-L6-v2) [32] and $\delta = 0.85$ is the similarity threshold. This embedding-based approach captures semantic equivalence that string matching would miss, preventing redundant triplets from entering the final knowledge graph.

3.7. Running Example

To provide concrete intuition for how AEVS operates, we present a complete running example using a representative input. This demonstration illustrates how each stage contributes to faithful extraction and hallucination detection. Consider the following input text:

“Steve Jobs co-founded Apple Inc. in Cupertino in 1976. He served as CEO until his resignation in 2011.”

This two-sentence passage contains multiple entities, relationships, and temporal attributes, presenting opportunities for both correct extraction and potential hallucination.

3.7.1. Running Example of Stage 1

The anchor discovery stage processes this text and identifies all semantically meaningful spans. Figure 3 visualizes the results. The system discovers a total of 10 anchors across three categories:

- **Entity anchors** $\mathcal{A}_E$ (four anchors): Steve Jobs @0–10 [person], Apple Inc. @22–32 [organization], Cupertino @36–45 [location], and CEO @68–71 [title]. These represent the named entities that can serve as subjects or objects in triplets.
- **Relation anchors** $\mathcal{A}_R$ (four anchors) with schema mappings: co-founded $\xrightarrow{\phi}$ founder @11–21, in $\xrightarrow{\phi}$ location @33–35, served as $\xrightarrow{\phi}$ position @58–67, and resignation $\xrightarrow{\phi}$ resignationYear @82–93. Note how diverse surface forms are normalized to canonical relation types via schema mapping.
- **Attribute anchors** $\mathcal{A}_T$ (two anchors): 1976 @49–53 [year] and 2011 @97–101 [year]. These temporal values will serve as objects in attribute triplets.

The complete anchor set $\mathcal{A} = \mathcal{A}_E \cup \mathcal{A}_R \cup \mathcal{A}_T$ now establishes the vocabulary of elements that can legitimately appear in extracted triplets. Any triplet element not traceable to this set is immediately suspicious.

(a) Input Text with Anchor Annotations

Steve Jobs@0–10 co-founded@11–21 Apple Inc.@22–32 in@33–35 Cupertino@36–45 in 1976@49–53. He served as@58–67 CEO@68–71 until his resignation@82–93 in 2011@97–101

Entity Anchor ($\mathcal{A}_E$) Relation Anchor ($\mathcal{A}_R$) Attribute Anchor ($\mathcal{A}_T$)

(b) Complete Anchor Set $\mathcal{A} = \mathcal{A}_E \cup \mathcal{A}_R \cup \mathcal{A}_T$

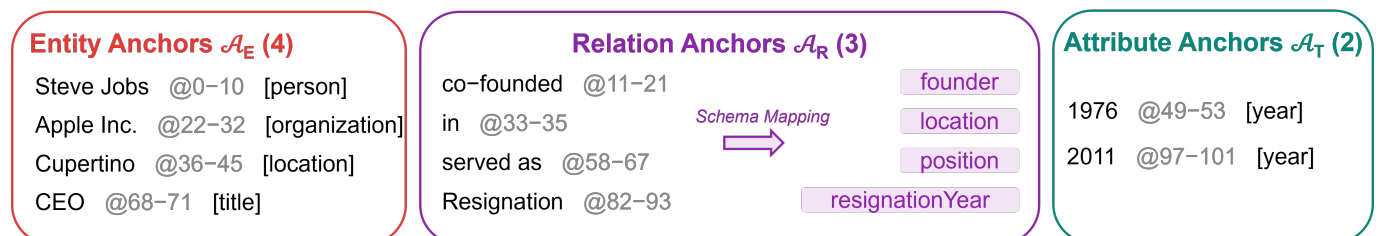


Figure 3. Stage 1 example: anchor discovery. (a) Input text with color-coded anchor annotations—entity (red), relation (purple), and attribute (teal)—each with character positions. (b) Complete anchor set $\mathcal{A} = \mathcal{A}_E \cup \mathcal{A}_R \cup \mathcal{A}_T$, with relation anchors showing schema mappings ϕ .

3.7.2. Running Example of Stage 2

With the anchor inventory established, the grounded extraction stage produces candidate triplets. Figure 4 visualizes this process. The LLM generates six triplets, each with claimed grounding information:

- τ_1 : (Steve Jobs, founder, Apple Inc.); fully grounded. The relation “founder” matches via schema mapping $\phi(\text{“co-founded”}) = \text{founder}$.
- τ_2 : (Apple Inc., location, Cupertino); fully grounded via direct anchor matches.
- τ_3 : (Steve Jobs, position, CEO); fully grounded. The relation uses $\phi(\text{“served as”}) = \text{position}$.
- τ_4 : (Steve Jobs, resignationYear, 2011); fully grounded via $\phi(\text{“resignation”}) = \text{resignationYear}$.

- τ_5 : (Steve Jobs, deathYear, 2011); *partially grounded*. The subject and object have valid groundings, but “deathYear” has no corresponding anchor or schema mapping.
- τ_6 : (Steve Jobs, nationality, American); *partially grounded*. Only the subject is grounded; neither “nationality” nor “American” appears anywhere in the text.

Triplets τ_5 and τ_6 illustrate common hallucination patterns; τ_5 misinterprets the year “2011” (which refers to resignation and not death), while τ_6 introduces external world knowledge not stated in the source text.

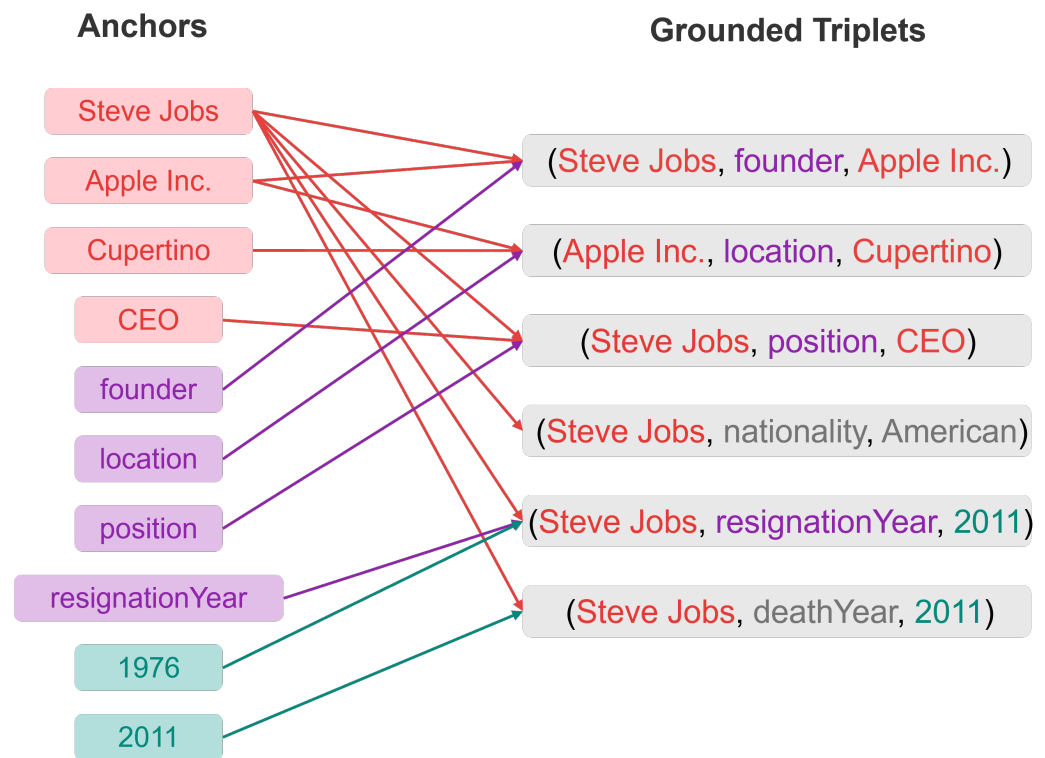


Figure 4. Stage 2 example: grounded extraction. Pink, purple, and teal boxes represent entity, relation, and attribute anchors, respectively. Matching colored arrows and text indicate grounding links; gray text marks ungrounded elements. Arrows show grounding links from anchors to triplet elements. Triplets τ_1 – τ_4 are fully grounded; τ_5 – τ_6 contain ungrounded elements (highlighted) that will require verification.

3.7.3. Running Example of Stage 3

The verification stage independently validates each triplet through restoration matching. Figure 5 demonstrates this process in detail.

For each triplet, we attempt to restore each element to the anchor set or original text, producing a restoration status vector $\rho = (\rho_s, \rho_r, \rho_o)$:

- τ_1 – τ_4 : All elements are successfully restored ($\rho = (1, 1, 1)$). For example, in τ_1 , “Steve Jobs” matches the entity anchor via m_{exact} , “founder” matches via m_{schema} through ϕ (“co-founded”), and “Apple Inc.” matches the entity anchor via m_{exact} . These triplets are **directly accepted** without LLM verification.
- τ_5 : $\rho = (1, 0, 1)$, where the subject “Steve Jobs” and object “2011” are restored, but the relation “deathYear” cannot be matched to any anchor or text span. This triplet is sent to **LLM verification**.
- τ_6 : $\rho = (1, 0, 0)$, where only the subject “Steve Jobs” is restored; both “nationality” and “American” fail restoration. This triplet is also sent to **LLM verification**.

LLM Verification. The verifier examines τ_5 and τ_6 with access to the original text and restoration status:

- τ_5 : The verifier correctly identifies that while “2011” appears in the text, it is associated with “resignation” and not death. The triplet conflates two different events. **Decision: REJECT** (hallucination, incorrect relationship inference).
- τ_6 : The verifier determines that neither “nationality” nor “American” appears in the source text. This represents external knowledge that may or may not be factually correct but is not extractable from this document. **Decision: REJECT** (hallucination, external knowledge injection).

Coverage-Aware Supplement. After verification and deduplication, we check the anchor coverage. The attribute anchor “1976” @49–53 remains unused in any accepted triplet. The supplement extraction is triggered, producing τ_7 : (Apple Inc., foundingYear, 1976). This triplet passes semantic filtering (not similar to any existing triplet) and is **accepted**.

Final Result. The complete knowledge graph contained five verified triplets: $\{\tau_1, \tau_2, \tau_3, \tau_4, \tau_7\}$. Two hallucinations (τ_5 and τ_6) were successfully detected and removed, while the coverage-aware supplement ensured that the founding year information was not missed.

(a) Restoration-based Verification Matching Results

Triplet	ρ_s (sub)	ρ_r (rel)	ρ_o (obj)	Fully Restored?	Decision	LLM Verification	Final Decision
τ_1 : (Steve Jobs, founder, Apple Inc.)	✓	✓	✓	Yes	ACCEPT	-	
τ_2 : (Apple Inc., location, Cupertino)	✓	✓	✓	Yes	ACCEPT	-	
τ_3 : (Steve Jobs, position, CEO)	✓	✓	✓	Yes	ACCEPT	-	
τ_4 : (Steve Jobs, resignationYear, 2011)	✓	✓	✓	Yes	ACCEPT	-	
τ_5 : (Steve Jobs, deathYear, 2011)	✓	✗	✓	No (2/3)	→ LLM	"INVALID - 2011 is resignation year, not death year"	REJECT
τ_6 : (Steve Jobs, nationality, American)	✓	✗	✗	No (1/3)	→ LLM	"INVALID - Neither appears in source text"	REJECT

(b) Coverage-aware Supplement Extraction

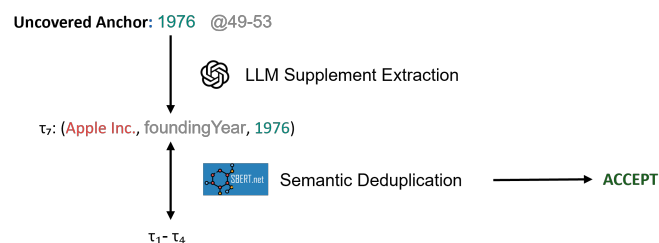


Figure 5. Stage 3 example: restoration-based verification and coverage-aware supplement. (a) Restoration matching results with element-level status (ρ_s, ρ_r, ρ_o). Green checkmarks and red crosses indicate successful and failed element restoration, respectively; green “ACCEPT” and red “REJECT” denote final decisions. Triplets τ_1 – τ_4 were fully restored and directly accepted. (b) LLM verification rejected τ_5 (deathYear unsupported) and τ_6 (hallucinated nationality). Coverage-aware supplement extracted τ_7 for unused anchor “1976”. Final result: 5 verified triplets.

3.8. Complete Algorithm

Algorithm 2 presents the complete AEVS pipeline integrating all three stages. The algorithm takes as input the source text $\mathcal{T}$, relation schema $\mathcal{R}$, and an LLM $\mathcal{M}$ and outputs a verified knowledge graph $\mathcal{K}^*$ with full provenance. The pipeline first discovers anchors and then extracts grounded triplets. The verification phase partitions triplets into fully restored (directly accepted) and partially restored (sent to LLM verification) sets based on restoration status. After deduplication, the optional coverage-aware supplement extracts

additional triplets for unused anchors, applies semantic filtering to remove redundancy, and merges them into the final knowledge graph.

Algorithm 2 AEVS: complete pipeline

Require: Text $\mathcal{T}$, Schema $\mathcal{R}$, LLM $\mathcal{M}$, Config $\mathcal{C}$

Ensure: Verified knowledge graph $\mathcal{K}^*$ with provenance

```

1:                                     ▷ Stage 1: Anchor Discovery
2:  $\mathcal{A} \leftarrow \text{DISCOVERANCHORS}(\mathcal{T}, \mathcal{R}, \mathcal{M})$                                      ▷ Algorithm 1
3:
4:                                     ▷ Stage 2: Grounded Extraction
5:  $\mathcal{K}_{\text{raw}} \leftarrow \text{EXTRACTGROUNDED}(\mathcal{T}, \mathcal{A}, \mathcal{M})$ 
6:
7:                                     ▷ Stage 3: Restoration-Based Verification and Coverage-Aware Supplement
8:  $\mathcal{K}_{\text{verified}}, \mathcal{K}_{\text{partial}} \leftarrow \emptyset, \emptyset$ 
9: for  $\tau \in \mathcal{K}_{\text{raw}}$  do
10:    $\rho(\tau) \leftarrow (\mathcal{H}_E(s, \mathcal{A}, \mathcal{T}).\rho, \mathcal{H}_R(r, \mathcal{A}_R, \mathcal{T}).\rho, \mathcal{H}_E(o, \mathcal{A}, \mathcal{T}).\rho)$ 
11:   if FULLYRESTORED( $\tau$ ) then
12:      $\mathcal{K}_{\text{verified}} \leftarrow \mathcal{K}_{\text{verified}} \cup \{\tau\}$                                      ▷ Direct Accept
13:   else
14:      $\mathcal{K}_{\text{partial}} \leftarrow \mathcal{K}_{\text{partial}} \cup \{(\tau, \rho(\tau))\}$ 
15:   end if
16: end for
17:
18:                                     ▷ LLM Verification for Partial Triplets
19:  $\mathcal{K}_{\text{llm\_ok}} \leftarrow \text{LLMVERIFY}(\mathcal{K}_{\text{partial}}, \mathcal{T}, \mathcal{M})$ 
20:  $\mathcal{K}_{\text{verified}} \leftarrow \mathcal{K}_{\text{verified}} \cup \mathcal{K}_{\text{llm\_ok}}$ 
21:
22:  $\mathcal{K}^* \leftarrow \text{DEDUPLICATE}(\mathcal{K}_{\text{verified}})$ 
23:
24:                                     ▷ Coverage-Aware Supplement
25: if  $\mathcal{C}.\text{enable\_supplement}$  then
26:    $\mathcal{A}_{\text{unused}} \leftarrow \{a \in \mathcal{A} : \nexists \tau \in \mathcal{K}^*, a \in \text{ANCHORS}(\tau)\}$ 
27:    $\mathcal{K}_{\text{supp}} \leftarrow \text{EXTRACTSUPPLEMENT}(\mathcal{A}_{\text{unused}}, \mathcal{K}^*, \mathcal{T}, \mathcal{M})$ 
28:    $\mathcal{K}_{\text{supp}} \leftarrow \{\tau \in \mathcal{K}_{\text{supp}} : \neg \exists \tau' \in \mathcal{K}^*, \text{SEMANTICDUP}(\tau, \tau'; \delta)\}$ 
29:    $\mathcal{K}^* \leftarrow \mathcal{K}^* \cup \mathcal{K}_{\text{supp}}$ 
30: end if
31:
32: return  $\mathcal{K}^*$ 

```

4. Experiments

In this section, we present a comprehensive empirical evaluation of AEVS across multiple dimensions. We first describe our experimental set-up, including the datasets, evaluation metrics, and baseline methods. We then report the main results comparing AEVS against both specialized trained models and existing LLM-based approaches, followed by detailed ablation studies that isolate the contribution of each component. We investigate the sensitivity of AEVS to temperature hyperparameters, providing practical guidance for deployment. We conduct a dedicated analysis of the anchor discovery quality and its relationship to downstream extraction performance. We quantify the computational cost of the AEVS pipeline and provide detailed analyses of the verification stage behavior, hallucination rates, and qualitative error patterns.

4.1. Experimental Set-Up

4.1.1. Datasets

We evaluated AEVS on three widely used knowledge graph construction benchmarks that collectively represent diverse characteristics in terms of annotation quality, relation

schema complexity, and domain coverage. Following the experimental protocol established in prior work [11], we selected datasets with rich relation vocabularies to better reflect the challenges encountered in real-world knowledge extraction scenarios.

- **WebNLG** [33] provides clean, human-annotated data for evaluation. We used the test split from WebNLG+2020 (v3.0), which contains 1165 text-triplet pairs spanning 159 unique relation types. A distinguishing characteristic of this dataset is the direct correspondence between text and triplets; annotators composed the natural language descriptions solely from the given triplets, ensuring that all textual content is explicitly grounded in the knowledge graph structure.
- **REBEL** [7] represents a large-scale, distantly supervised benchmark constructed by aligning Wikipedia abstracts with Wikidata entities. To ensure fair comparison with EDC [11], we used the identical 1000 text-triplet pairs sampled by the original EDC work from the test partition, resulting in a schema with 200 distinct relation types. The distant supervision methodology, while enabling large-scale annotation, introduces inherent noise and incompleteness; not all triplets inferable from the text are necessarily present in the reference annotations.
- **Wiki-NRE** [34] provides another perspective on distantly supervised KGC with a more constrained relation schema of 45 unique relation types. Similarly, we used the same 1000 text-triplet pairs sampled by EDC from the test split to ensure consistent comparison. The smaller relation vocabulary compared with REBEL allowed us to examine whether the benefits of AEVS generalize across different levels of schema complexity.

We deliberately excluded datasets with extremely limited relation types, such as ADE (1 relation type), SciERC (7 relation types), and CoNLL04 (4 relation types), as they do not adequately capture the diversity of relations encountered in practical knowledge graph construction tasks.

4.1.2. Evaluation Metrics

Following the evaluation protocol established in prior work [11,27], we employed the official WebNLG evaluation script [33] to compute token-level precision (P), recall (R), and F1 scores. The evaluation considers three increasingly stringent matching criteria that capture different aspects of extraction quality:

- **Partial matching** computes token overlap between candidate and reference triplets without requiring exact correspondence. This metric is lenient toward minor surface-form variations while still penalizing fundamentally incorrect extractions, providing a useful measure of semantic overlap even when exact string matching fails.
- **Strict matching** extends partial matching by additionally requiring that the element types (subject, predicate, and object) are correctly identified. A candidate triplet receives credit only when the matching tokens also occupy the correct structural positions within the triplet.
- **Exact matching** represents the most stringent criterion, requiring complete token-level matching between candidate and reference triplets. While this metric may underestimate performance when valid paraphrases exist, it provides an unambiguous measure of extraction quality and is particularly relevant for applications requiring exact schema conformance.

4.1.3. Baselines

We compare AEVS against representative methods from three distinct paradigms to provide a comprehensive assessment of its effectiveness:

- **Specialized Trained Models:** This category represents the state of the art in supervised knowledge graph construction, where models are fine-tuned on domain-specific training data. We included two representative systems: **REGEN** [27], which leverages pretrained T5 with reinforcement learning for bidirectional text-to-graph generation and achieves state-of-the-art performance on WebNLG, and **GenIE** [8], a BART-based model employing constrained generation strategies to ensure schema conformance, which leads performance on REBEL and Wiki-NRE.
- **LLM-Based Frameworks:** We ran a comparison against **EDC** [11], the current state-of-the-art LLM-based KGC framework. EDC implements a three-phase extract-define-canonicalize pipeline that performs open information extraction followed by post hoc schema canonicalization. Due to the cost constraints of running the multi-stage EDC pipeline across multiple LLMs and large-scale datasets, we implemented EDC using GPT-4o-mini as the backbone model, which also corresponds to one of our AEVS configurations for direct comparison under identical model conditions.
- **Direct LLM Prompting:** We established baseline performance by evaluating direct triplet extraction using the same backbone LLMs employed in AEVS without any structured extraction framework. This comparison quantified the improvement attributable to the AEVS framework design itself, independent of underlying model capabilities.

4.1.4. Implementation Details

We evaluated AEVS with four large language models representing diverse model families and capabilities: GPT-4o-mini and GPT-5.1 from OpenAI, Claude 4.5 Haiku from Anthropic, and Gemini 2.5 Flash from Google. This selection spanned different architectural designs and training methodologies, allowing us to assess the generalizability of AEVS across model providers. For semantic deduplication during anchor discovery, we employed the all-MiniLM-L6-v2 sentence transformer [32] with a similarity threshold of $\delta = 0.85$. All LLM inference used default temperature settings unless otherwise specified in the temperature sensitivity analysis.

4.2. Main Results

Tables 2–4 present the comprehensive results across all three datasets under the three evaluation criteria. We organized the results by method category to facilitate systematic comparison.

Table 2. Results on **WebNLG** (159 relation types). Best results are **bolded**, and second-best results are underlined. [†]: specialized trained model; [‡]: LLM-based framework.

Method	Partial			Strict			Exact		
	P	R	F1	P	R	F1	P	R	F1
REGEN [†]	0.755	0.788	0.767	0.713	0.735	0.720	0.714	0.738	0.723
Direct (GPT-4o-mini)	0.580	0.620	0.594	0.450	0.473	0.458	0.491	0.515	0.499
Direct (GPT-5.1)	0.607	0.654	0.624	0.468	0.496	0.479	0.509	0.538	0.520
Direct (Claude 4.5 Haiku)	0.590	0.628	0.603	0.465	0.486	0.473	0.506	0.528	0.514
Direct (Gemini 2.5 Flash)	0.586	0.629	0.602	0.448	0.474	0.458	0.487	0.513	0.496
EDC [‡] (GPT-4o-mini)	0.808	0.832	0.817	0.748	0.764	0.753	0.759	0.775	0.765
AEVS (GPT-4o-mini)	0.817	0.838	0.824	0.756	0.770	0.760	0.780	0.794	0.785
AEVS (GPT-5.1)	0.827	0.853	0.836	0.762	0.779	0.768	0.773	0.790	0.779
AEVS (Claude 4.5 Haiku)	<u>0.830</u>	<u>0.852</u>	<u>0.841</u>	<u>0.774</u>	<u>0.788</u>	<u>0.781</u>	<u>0.787</u>	<u>0.802</u>	<u>0.794</u>
AEVS (Gemini 2.5 Flash)	0.863	0.886	0.874	0.795	0.810	0.802	0.821	0.836	0.828

Table 3. Results on **REBEL** (200 relation types). Best results are **bolded**, and second-best are underlined. [†]: specialized trained model; [‡]: LLM-based framework.

Method	Partial			Strict			Exact		
	P	R	F1	P	R	F1	P	R	F1
GenIE [†]	0.381	0.391	0.385	0.353	0.356	0.362	0.362	0.369	0.364
Direct (GPT-4o-mini)	0.503	0.535	0.515	0.412	0.431	0.419	0.439	0.460	0.446
Direct (GPT-5.1)	0.606	0.632	0.616	0.520	0.537	0.526	0.544	0.562	0.551
Direct (Claude 4.5 Haiku)	0.513	0.573	0.535	0.417	0.460	0.433	0.448	0.492	0.464
Direct (Gemini 2.5 Flash)	0.559	0.594	0.571	0.477	0.500	0.486	0.503	0.521	0.512
EDC [‡] (GPT-4o-mini)	0.687	0.705	0.694	0.619	0.630	0.623	0.638	0.650	0.643
AEVS (GPT-4o-mini)	0.699	0.719	0.706	0.632	0.643	0.636	0.651	0.663	0.655
AEVS (GPT-5.1)	<u>0.736</u>	<u>0.747</u>	<u>0.740</u>	<u>0.661</u>	<u>0.667</u>	<u>0.663</u>	<u>0.680</u>	<u>0.687</u>	<u>0.683</u>
AEVS (Claude 4.5 Haiku)	0.793	0.803	0.797	0.730	0.736	0.732	0.755	0.762	0.757
AEVS (Gemini 2.5 Flash)	0.721	0.746	0.730	0.634	0.646	0.639	0.655	0.670	0.660

Table 4. Results on **Wiki-NRE** (45 relation types). Best results are **bolded**, and second-best are underlined. [†]: specialized trained model; [‡]: LLM-based framework.

Method	Partial			Strict			Exact		
	P	R	F1	P	R	F1	P	R	F1
GenIE [†]	0.482	0.486	0.484	0.462	0.464	0.463	0.477	0.479	0.478
Direct (GPT-4o-mini)	0.524	0.579	0.541	0.435	0.474	0.447	0.496	0.536	0.508
Direct (GPT-5.1)	0.546	0.606	0.564	0.458	0.506	0.473	0.521	0.569	0.536
Direct (Claude 4.5 Haiku)	0.534	0.600	0.556	0.451	0.502	0.468	0.509	0.561	0.527
Direct (Gemini 2.5 Flash)	0.528	0.590	0.549	0.448	0.496	0.464	0.504	0.552	0.520
EDC [‡] (GPT-4o-mini)	0.722	0.734	0.727	0.688	0.697	0.692	0.699	0.708	0.702
AEVS (GPT-4o-mini)	0.794	0.813	0.801	0.759	0.772	0.764	0.769	0.783	0.774
AEVS (GPT-5.1)	0.782	0.791	0.785	0.748	0.755	0.751	0.761	0.768	0.764
AEVS (Claude 4.5 Haiku)	<u>0.820</u>	<u>0.825</u>	<u>0.822</u>	<u>0.799</u>	<u>0.803</u>	<u>0.800</u>	<u>0.807</u>	<u>0.810</u>	<u>0.808</u>
AEVS (Gemini 2.5 Flash)	0.877	0.882	0.879	0.858	0.861	0.859	0.861	0.864	0.862

4.2.1. Comparison with Specialized Trained Models

The experimental results demonstrate that AEVS consistently outperformed specialized trained models across all datasets and evaluation criteria, despite not requiring any task-specific training. On WebNLG, AEVS with Gemini 2.5 Flash achieved substantial improvements over REGEN across all three metrics: **+0.107** in Partial F1 (0.874 vs. 0.767), **+0.082** in Strict F1 (0.802 vs. 0.720), and **+0.105** in Exact F1 (0.828 vs. 0.723). These consistent gains across different matching criteria indicate that AEVS not only captured more correct content but also maintained proper structural organization of triplet elements.

The performance gap widened substantially on the distantly supervised datasets. On REBEL, AEVS with Claude 4.5 Haiku outperformed GenIE by **+0.412** in Partial F1 (0.797 vs. 0.385), **+0.370** in Strict F1 (0.732 vs. 0.362), and **+0.393** in Exact F1 (0.757 vs. 0.364). Similarly, on Wiki-NRE, AEVS with Gemini 2.5 Flash achieved improvements of **+0.395**, **+0.396**, and **+0.384** under the partial, strict, and exact criteria, respectively. We attribute GenIE's relatively weak performance to its constrained generation strategy, which enforces strict schema conformance but struggles to handle literals such as numerical values, dates, and other attribute expressions that do not conform to predefined entity types. In contrast, AEVS's flexible anchor discovery mechanism naturally accommodates diverse surface forms, including literals, by treating all salient text spans as potential anchors without imposing rigid type constraints.

4.2.2. Improvement over Direct Prompting

A critical validation of the AEVS framework comes from comparison against direct prompting using identical backbone LLMs. This comparison isolates the contribution of our anchor-grounded extraction paradigm from the capabilities of the underlying language model. The results consistently show substantial improvements across all model, dataset, and metric combinations.

Taking GPT-4o-mini as an example, AEVS achieved comprehensive improvements on WebNLG: **+0.230** in Partial F1 (0.824 vs. 0.594), **+0.302** in Strict F1 (0.760 vs. 0.458), and **+0.286** in Exact F1 (0.785 vs. 0.499). The larger gains observed under the strict and exact criteria are particularly noteworthy, as they indicate that AEVS not only improved content accuracy but also substantially enhanced the structural correctness of extracted triplets. Similar patterns were observed on REBEL (+0.191, +0.217, and +0.209, respectively) and Wiki-NRE (+0.260, +0.317, and +0.266, respectively).

Across all tested LLMs, the improvements remained consistent. GPT-5.1 showed partial, strict, and exact improvements of +0.212, +0.289, +0.259 on WebNLG, +0.124, +0.137, and +0.132 on REBEL, and +0.221, +0.278, and +0.228 on Wiki-NRE, respectively. Claude 4.5 Haiku achieved gains of +0.238, +0.308, and +0.280 on WebNLG, +0.262, +0.299, and +0.293 on REBEL, and +0.266, +0.332, and +0.281 on Wiki-NRE, respectively. Gemini 2.5 Flash demonstrated improvements of +0.272, +0.344, and +0.332 on WebNLG, +0.159, +0.153, and +0.148 on REBEL, and +0.330, +0.395, and +0.342 on Wiki-NRE, respectively. The consistency of these improvements across different model families and evaluation criteria confirms that the performance gains stemmed from the AEVS framework design rather than any particular model's inherent capabilities.

4.2.3. Comparison with LLM-Based Frameworks

To ensure a fair comparison with EDC, the current state-of-the-art LLM-based framework, we compared the performance using the identical backbone model (GPT-4o-mini). This controlled comparison isolated the effectiveness of the extraction framework design from the influence of underlying model capabilities.

On WebNLG, AEVS outperformed EDC across all metrics under identical model conditions: **+0.007** in Partial F1 (0.824 vs. 0.817), **+0.007** in Strict F1 (0.760 vs. 0.753), and **+0.020** in Exact F1 (0.785 vs. 0.765). While the improvements on this clean, human-annotated dataset were modest, they demonstrate that AEVS achieved competitive performance without sacrificing accuracy.

The advantages of AEVS became more pronounced on the distantly supervised datasets. On REBEL, AEVS surpassed EDC by **+0.012** in Partial F1 (0.706 vs. 0.694), **+0.013** in Strict F1 (0.636 vs. 0.623), and **+0.012** in Exact F1 (0.655 vs. 0.643). On Wiki-NRE, AEVS achieved substantially larger margins: **+0.074** in Partial F1 (0.801 vs. 0.727), **+0.072** in Strict F1 (0.764 vs. 0.692), and **+0.072** in Exact F1 (0.774 vs. 0.702). The consistent improvements across all matching criteria suggest that AEVS not only extracted more accurate content but also better preserved the structural integrity of triplets.

These results suggest that the anchor-grounded extraction paradigm employed by AEVS offers fundamental advantages over EDC's canonicalization-based approach. While EDC first performs open information extraction and then attempts to map the extracted relations to the target schema, this two-stage process can introduce errors during canonicalization, particularly when the extracted relations do not cleanly align with predefined schema categories. AEVS avoids this issue by constraining extraction to text-grounded anchors from the outset, ensuring that all extracted triplets maintain explicit provenance to source text spans.

4.2.4. Analysis of Backbone LLM Effects

While AEVS improved performance across all tested backbone LLMs, the magnitude of improvement and absolute performance levels varied across the models and datasets. Notably, no single model dominated across all experimental conditions. Gemini 2.5 Flash achieved the best results on WebNLG (Partial/Strict/Exact F1: 0.874/0.802/0.828) and Wiki-NRE (Partial/Strict/Exact F1: 0.879/0.859/0.862), while Claude 4.5 Haiku led on REBEL (Partial/Strict/Exact F1: 0.797/0.732/0.757). This variation suggests that model characteristics interacted with dataset properties in nuanced ways that merit further investigation.

We observe that the models exhibiting stronger performance under direct prompting did not always maintain their relative ranking when augmented with AEVS. For instance, GPT-5.1 outperformed Claude 4.5 Haiku under direct prompting on REBEL across all metrics (Partial F1: 0.616 vs. 0.535; Strict F1: 0.526 vs. 0.433; Exact F1: 0.551 vs. 0.464), but this ordering was reversed with AEVS (Partial F1: 0.740 vs. 0.797; Strict F1: 0.663 vs. 0.732; Exact F1: 0.683 vs. 0.757). This finding indicates that different models may benefit differently from the structured extraction guidance provided by AEVS, potentially due to varying instruction-following capabilities or generation characteristics.

4.2.5. Dataset-Specific Observations

An interesting pattern emerged when examining the relative improvements across datasets. AEVS showed notably stronger relative improvements on REBEL and Wiki-NRE compared with WebNLG. We hypothesize that this stems from the incomplete annotations characteristic of distantly supervised datasets. In REBEL and Wiki-NRE, valid triplets inferable from the text are often missing from the reference annotations due to the limitations of distant supervision. AEVS, by thoroughly identifying text-grounded anchors and systematically exploring their combinations, may extract valid triplets that happen to be absent from the reference set, which would not be fully reflected in automatic evaluation metrics.

4.2.6. Evaluation Limitations on Distantly Supervised Datasets

An important caveat applies to the evaluation on REBEL and Wiki-NRE. Both datasets rely on distant supervision, where triplet annotations are automatically generated by aligning knowledge base entries to text rather than through human judgment. This methodology introduces two well-known biases. First, the annotations are *incomplete*; valid triplets inferable from the text may be absent from the reference set because the corresponding knowledge base entries do not exist or the alignment heuristic fails to match them. As a consequence, higher-recall methods—including AEVS—may extract valid but unannotated triplets that are counted as false positives, potentially understating true precision. Second, the annotations may contain *noise*; some reference triplets may be weakly supported by the text due to imprecise alignment, inflating recall for methods that coincidentally reproduce such noisy annotations.

These biases affect all methods equally and do not invalidate relative comparisons. However, they imply that absolute F1 scores on REBEL and Wiki-NRE should be interpreted cautiously, as they may underestimate the extraction quality of high-recall approaches. Our qualitative error analysis (Section 4.9) partially addresses this limitation by manually inspecting representative failure cases and confirming that many detected “errors” correspond to unsupported relation labels rather than genuinely incorrect extractions. A more rigorous mitigation—human annotation of a held-out subset—remains an important direction for future work.

4.3. Ablation Study

To understand the contribution of each component to the overall performance of AEVS, we conducted systematic ablation experiments. Starting from the direct prompting baseline, we progressively added each component:

- **+AE**: Anchor-based extraction, comprising Stage 1 (Anchor Discovery) and Stage 2 (Grounded Extraction).
- **+RV**: Restoration-based verification.
- **+SE**: Coverage-aware supplement extraction.

This incremental evaluation allowed us to isolate the marginal contribution of each design decision. Figures 6–8 visualize the progressive performance improvements under the three matching criteria. In these figures, the background colors distinguish between different stages; gray represents the direct baseline, purple indicates the addition of anchor-based extraction (+AE), green shows the further addition of restoration-based verification (+RV), and pink marks the complete AEVS system with coverage-aware supplement extraction (+SE).

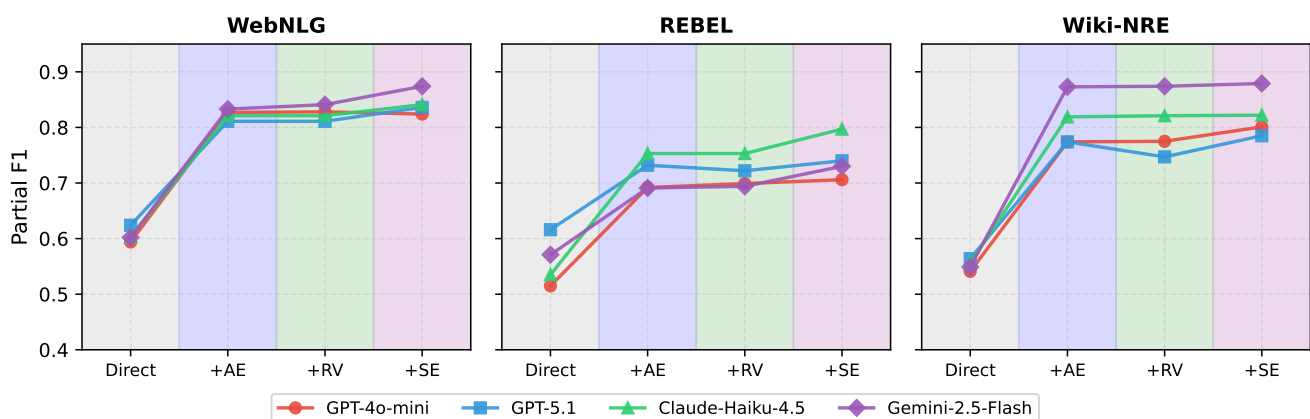


Figure 6. Ablation study results under partial matching. Background colors indicate progressive component additions: gray (direct baseline), purple (+AE), green (+RV), and pink (+SE). The most substantial improvement occurred upon adding anchor-based extraction, validating the effectiveness of text-grounded constraint in reducing hallucinated outputs.

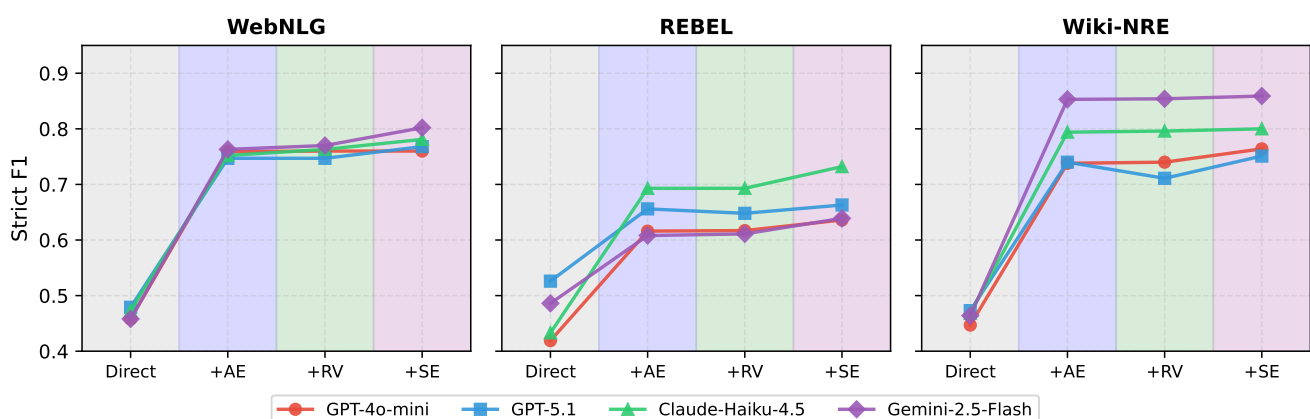


Figure 7. Ablation study results under strict matching.

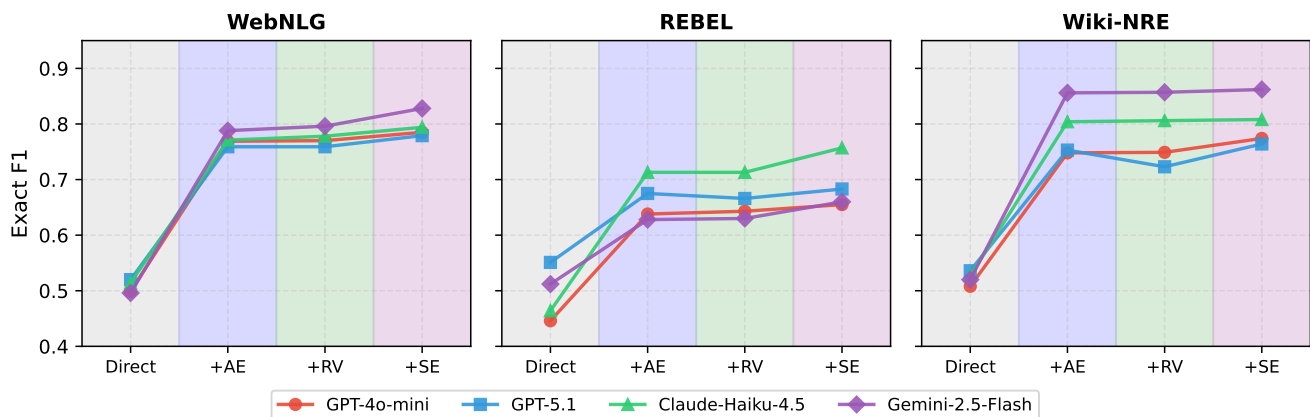


Figure 8. Ablation study results under exact matching.

4.3.1. The Central Role of Anchor-Based Extraction

The transition from direct prompting to +AE yielded by far the largest performance improvement across all experimental conditions and evaluation criteria. On WebNLG, adding anchor-based extraction improved the Partial F1 by +0.188 to +0.233, depending on the backbone LLM. The gains were even more pronounced under strict matching (+0.269 to +0.305) and exact matching (+0.240 to +0.292). On REBEL, improvements ranged from +0.117 to +0.218 under partial matching, +0.123 to +0.260 under strict matching, and +0.116 to +0.249 under exact matching. Wiki-NRE showed the largest gains: +0.210 to +0.324 (partial), +0.267 to +0.389 (strict), and +0.217 to +0.336 (exact). The consistently larger improvements under the strict and exact criteria indicate that anchor-based extraction not only captured more correct content but also substantially improved the structural organization of triplet elements.

The mechanism underlying this improvement relates directly to the hallucination problem that plagues direct prompting approaches. When prompted to extract triplets without structural guidance, language models can generate plausible-sounding but unsupported relational facts, entities that seem reasonable given the topic but lack explicit textual evidence or relations that represent commonsense inferences rather than stated information. The anchor discovery stage eliminates this failure mode by first identifying all salient text spans and then requiring subsequent extraction to select and combine only from this discovered vocabulary. This design ensures that every element of an extracted triplet can be traced back to specific tokens in the source text, effectively preventing the introduction of fabricated entities and relations.

4.3.2. Verification as a Traceability Guarantee

The +RV stage (restoration-based verification) contributed relatively modest changes to the aggregate F1 metrics, typically ranging from -0.010 to $+0.011$, with occasional larger decreases observed for certain model-dataset combinations (e.g., GPT-5.1 on Wiki-NRE showed decreases of 0.028–0.030 from +AE to +RV across the three metrics). At first glance, this might suggest that verification adds limited value. However, this interpretation misses the primary purpose of restoration-based verification.

The minimal F1 impact of +RV actually reflects a positive finding; it indicates that most triplets extracted under anchor constraints are already traceable to source text. The anchor-based extraction stage has already performed the heavy lifting of eliminating hallucinated content. What restoration-based verification adds is not primarily improved aggregate metrics but rather an explicit provenance guarantee. Triplets that pass the restoration matching process are guaranteed to have explicit textual support; the verification mechanism can reconstruct a path from each triplet element back to specific text spans. This

property is essential for high-stakes applications in domains such as healthcare, legal, and scientific research, where users must be able to audit extraction outputs and verify their textual grounding.

4.3.3. Coverage Enhancement Through Supplement Extraction

The final component, coverage-aware supplement extraction (+SE), contributed improvements typically ranging from +0.001 to +0.045, with occasional marginal decreases in specific configurations. The magnitude of improvement correlated with the characteristics of both the dataset and the backbone model. Larger gains were consistently observed on distantly supervised datasets: REBEL showed improvements of up to +0.045 with Claude 4.5 Haiku, while Wiki-NRE showed gains up to +0.041 with GPT-5.1.

This pattern reflects the design purpose of supplement extraction. By identifying anchors that were discovered but not utilized in any extracted triplet, the supplement stage provides additional opportunities to extract valid relations that were missed in the initial grounded extraction phase. On distantly supervised datasets where annotations are incomplete, this mechanism recovers valid triplets that would otherwise be lost. The supplement stage is particularly effective for attribute anchors, namely numerical values, dates, quantities, and similar literals that may be overlooked during initial extraction but represent important relational information when systematically processed.

A natural concern is whether supplement extraction risks reintroducing hallucinations by making additional LLM calls in a high-recall setting. Our verification stage analysis (Section 4.7) addresses this directly; supplement-generated triplets undergo the same hierarchical restoration and verification process as the initial triplets, and the hallucination analysis (Section 4.8) confirms that the overall hallucination rates remain low across all configurations. In particular, Claude 4.5 Haiku—the model with the highest supplement volumes (from +510 to +1016 triplets)—maintained the lowest hallucination rates (0.23–0.44%), demonstrating that high supplement output does not correlate with increased hallucination risk when the backbone model exhibits strong instruction adherence.

4.4. Temperature Sensitivity Analysis

The temperature hyperparameter governs the randomness of language model outputs and can significantly affect extraction behavior. To provide practical guidance for AEVS deployment, we conducted systematic experiments investigating how temperature settings affect performance across different pipeline stages. Due to computational cost constraints, we performed these experiments on a 10% random sample of each dataset, using the best-performing backbone LLM identified in the main results for each dataset: Gemini 2.5 Flash for WebNLG and Wiki-NRE and Claude 4.5 Haiku for REBEL.

4.4.1. Experimental Design

AEVS involves four temperature parameters corresponding to different pipeline stages, where τ_1 controls the anchor discovery stage (Stage 1), τ_2 governs the grounded extraction stage (Stage 2), τ_v sets the temperature for restoration-based verification, and τ_s determines the behavior of coverage-aware supplement extraction. We structured our analysis into two parts based on the functional grouping of these stages.

For Stages 1 and 2 (Anchor Discovery and Grounded Extraction), we evaluated all combinations of $\tau_1, \tau_2 \in \{0, 0.1, 0.2, 0.3, 0.5, 0.8\}$, resulting in 36 configurations per dataset. These stages involved generative processes where temperature could meaningfully affect the diversity and creativity of outputs.

For Stage 3 (Verification and Supplement), we used the optimal Stage 1 and 2 temperature settings identified above and then evaluated all combinations of τ_v, τ_s over the same

temperature range. This sequential design allowed us to isolate the effects of verification and supplement temperatures from the extraction stages.

4.4.2. Results for Stages 1 and 2

Figures 9–11 present heat map visualizations of how Stage 1 and 2's temperature settings affected performance under the three matching criteria. Each heat map displays the F1 scores as a function of τ_1 (vertical axis) and τ_2 (horizontal axis), with warmer colors indicating better performance.

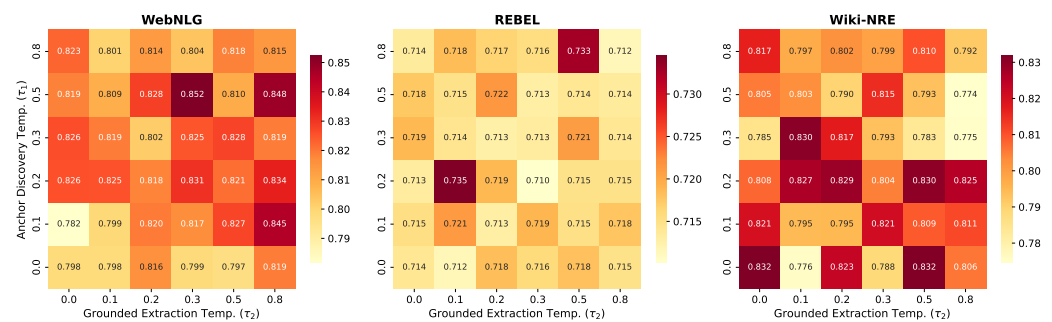


Figure 9. Temperature sensitivity analysis for Stages 1 and 2 under partial matching. Heat maps show F1 scores across temperature combinations for each dataset. Results based on 10% data samples using the best-performing model for each dataset.

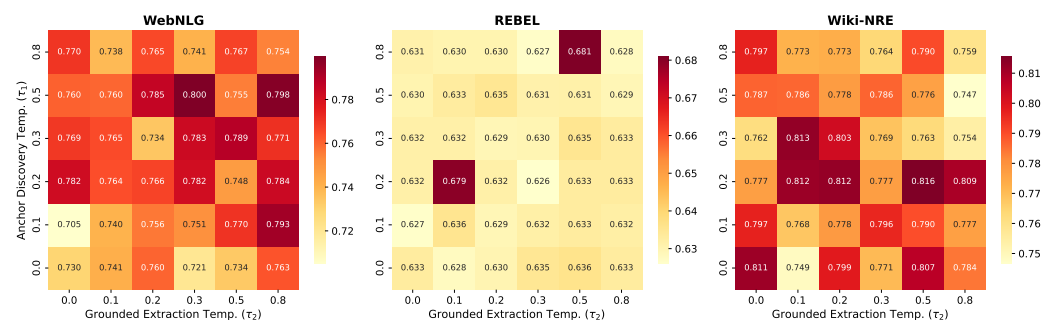


Figure 10. Temperature sensitivity analysis for Stages 1 and 2 under strict matching.

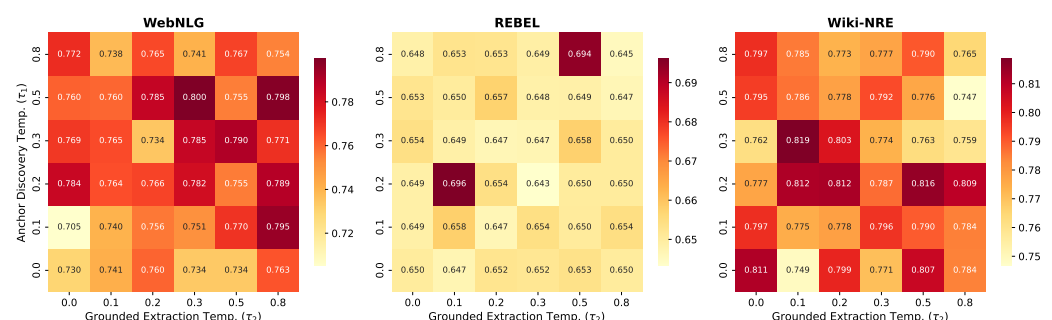


Figure 11. Temperature sensitivity analysis for Stages 1 and 2 under exact matching.

The heat maps reveal that optimal temperature configurations varied substantially across the datasets. By examining performance across all three matching criteria, we identified the following optimal configurations:

- **WebNLG:** The configuration $\tau_1 = 0.5$, $\tau_2 = 0.3$ achieved the best overall performance across all three metrics, with a Partial F1 of 0.852, Strict F1 of 0.800, and Exact F1 of 0.800. This suggests that moderate randomness benefits extraction from this clean, human-annotated dataset, where the human-composed text exhibits lexical variety

that higher temperatures help capture. The Partial F1 ranged from 0.782 to 0.852 across all configurations, showing a variance of approximately 0.071.

- **REBEL**: Optimal performance was achieved at $\tau_1 = 0.2$, $\tau_2 = 0.1$, yielding a Partial F1 of 0.735, Strict F1 of 0.679, and Exact F1 of 0.696. The noisier annotation environment of this distantly supervised dataset favors more deterministic generation that produces conservative, high-confidence extractions. The temperature sensitivity was relatively low, with the Partial F1 ranging only from 0.710 to 0.735 (variance of 0.024).
- **Wiki-NRE**: The optimal configuration varied slightly across the metrics. For the Partial F1, $\tau_1 = 0$, $\tau_2 = 0$ achieved the highest score (0.832), while for the Strict F1, $\tau_1 = 0.2$, $\tau_2 = 0.5$ performed best (0.816). Considering the average across all three metrics, $\tau_1 = 0.3$, $\tau_2 = 0.1$ provided the best balance (partial: 0.830; strict: 0.813; exact: 0.819). The dataset generally prefers lower temperatures, consistent with its smaller relation vocabulary (45 types).

4.4.3. Results for Stage 3

Figures 12–14 present analogous heat maps for Stage 3 temperature sensitivity, evaluated using the optimal Stage 1 and 2 configurations identified above. A striking visual difference from the Stage 1 and 2 results is immediately apparent; the Stage 3 heat maps exhibit much more uniform color distributions, indicating substantially lower sensitivity to temperature settings.

The Stage 3 results exhibited much more uniform performance across temperature settings compared with Stages 1 and 2. When examining the heat maps across all three matching criteria, we identify the following optimal configurations:

- **WebNLG**: The configuration $\tau_v = 0.1$, $\tau_s = 0$ achieved the best overall performance (Partial F1 = 0.877; Strict F1 = 0.823; Exact F1 = 0.826). However, the performance variance across all temperature combinations was minimal, with the Partial F1 ranging only from 0.862 to 0.877 (variance of 0.015).
- **REBEL**: Optimal performance was achieved at $\tau_v = 0.2$, $\tau_s = 0.3$ (Partial F1 = 0.769; Strict F1 = 0.709; Exact F1 = 0.729). The variance remained low at approximately 0.019 (Partial F1 ranging from 0.749 to 0.769).
- **Wiki-NRE**: The configuration $\tau_v = 0$, $\tau_s = 0.1$ achieved the best results across all three metrics (Partial F1 = 0.842; Strict F1 = 0.830; Exact F1 = 0.830), consistent with the dataset's preference for deterministic generation observed in Stages 1 and 2. The variance was only 0.013.

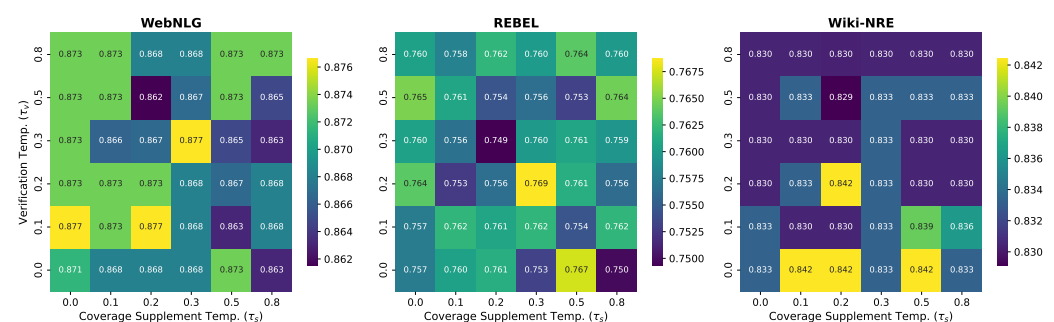


Figure 12. Temperature sensitivity analysis for Stage 3 under partial matching. The relatively uniform color distribution indicates that the verification and supplement stages are less sensitive to temperature settings compared with the extraction stages.

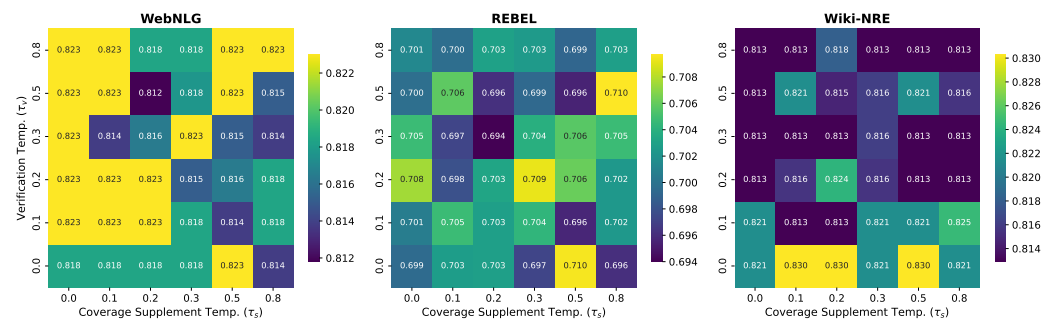


Figure 13. Temperature sensitivity analysis for Stage 3 under strict matching.

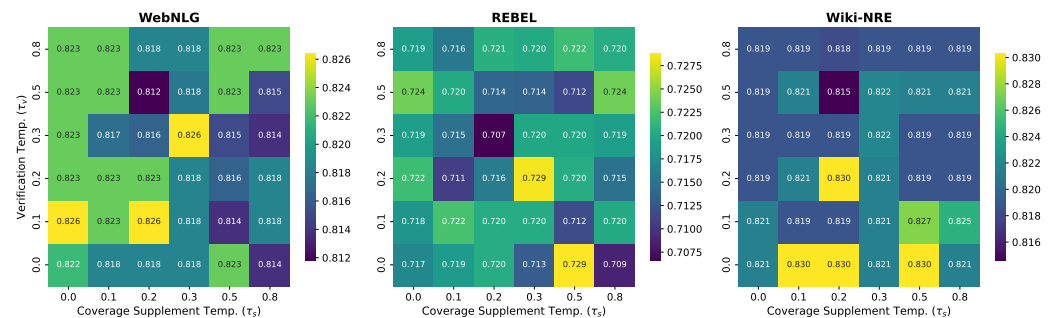


Figure 14. Temperature sensitivity analysis for Stage 3 under exact matching.

4.4.4. Key Findings

Our temperature sensitivity analysis, conducted on 10% data samples using the best-performing model for each dataset, yielded several findings with practical implications for AEVS deployment.

The extraction stages exhibited higher temperature sensitivity than the verification stages. The F1 variance across temperature settings in Stages 1 and 2 was substantial. On WebNLG, the Partial F1 ranged from 0.782 to 0.852 (0.071 variance); on Wiki-NRE, it ranged from 0.774 to 0.832 (0.057 variance); and even on REBEL, where the sensitivity was lowest, the range was from 0.710 to 0.735 (0.024 variance). In contrast, the Stage 3 variance remained below 0.020 across all datasets and metrics. This asymmetry aligns with the functional design of these stages; extraction requires generative exploration to identify diverse anchors and construct triplet combinations from discovered elements, making it inherently sensitive to the sampling temperature. In contrast, verification primarily involves deterministic matching operations—checking whether triplet elements can be restored to source anchors—where temperature has a limited influence on the binary accept or reject decision.

No universal optimal temperature existed across the datasets. The optimal temperature configuration depends on the dataset characteristics in interpretable ways. Distantly supervised datasets with potential annotation noise (REBEL or Wiki-NRE) generally prefer lower temperatures that yield conservative, high-confidence extractions. Clean, human-annotated datasets (WebNLG) with diverse surface expressions benefit from moderate randomness that explores alternative phrasings. Importantly, the optimal configurations identified under partial matching generally also performed well under the strict and exact criteria, suggesting that temperature tuning does not require separate optimization for each metric.

4.5. Anchor Discovery Quality Analysis

The ablation results in Section 4.3 identified anchor discovery as the foundation upon which all subsequent stages depend. A natural question arises: *How accurate is anchor discovery itself, and to what extent do anchor-level errors propagate to the final extraction results?* To answer this, we conducted a dedicated analysis of anchor discovery quality and its downstream impact across all model-dataset configurations.

4.5.1. Evaluation Methodology

Directly evaluating anchor quality requires a ground-truth anchor set, which is not available in standard KGE benchmarks. We therefore constructed *pseudo-gold anchors* by decomposing the gold triplet annotations. Subject and object strings were treated as gold entity anchors, while predicate strings served as gold relation anchors. We then computed the micro-averaged precision, recall, and F1 score between the discovered anchors $\mathcal{A}$ and pseudo-gold anchors $\mathcal{A}_{\text{gold}}$ separately for entities and relations. Matching employs the normalization function η defined in Section 3.6, augmented with fuzzy matching (token-level Jaccard overlap ≥ 0.6 and character-level similarity ratio ≥ 0.8) to accommodate minor surface-form variations such as determiners (“the United States” vs. “United States”) and camelCase splitting (“birthPlace” vs. “birth place”).

We additionally report the *entity miss rate* and *relation miss rate*, defined as the fraction of gold entities or relations, respectively, that failed to match any discovered anchor. These miss rates directly quantify the risk of irrecoverable recall loss due to anchor omissions. We note that this evaluation provides a *conservative* estimate of anchor quality; the pseudo-gold set was derived from gold annotations that may themselves be incomplete (particularly for REBEL and Wiki-NRE), and the LLM may have discovered valid anchors corresponding to correct but unannotated triplets, which would be counted as false positives under this protocol.

4.5.2. Results

Table 5 presents the anchor discovery quality metrics across all configurations, while Table 6 provides complementary statistics, including the average anchor counts, sample-level F1 distributions, and miss rates.

Table 5. Anchor discovery quality evaluation. Pseudo-gold anchors were derived from gold triplet annotations. Metrics are micro-averaged precision (P), recall (R), and F1 score for entity and relation anchors separately and overall.

Dataset	Model	Entity			Relation			Overall		
		P	R	F1	P	R	F1	P	R	F1
WebNLG	GPT-4o-mini	0.592	0.783	0.674	0.781	0.618	0.690	0.658	0.725	0.690
	GPT-5.1	0.578	0.821	0.678	0.752	0.783	0.767	0.639	0.808	0.714
	Claude-4.5-Haiku	0.586	0.843	0.691	0.779	0.762	0.770	0.654	0.815	0.726
	Gemini-2.5-Flash	0.612	0.867	0.718	0.803	0.794	0.798	0.679	0.842	0.752
REBEL	GPT-4o-mini	0.721	0.862	0.785	0.453	0.341	0.389	0.612	0.602	0.607
	GPT-5.1	0.618	0.937	0.745	0.581	0.468	0.518	0.604	0.706	0.651
	Claude-4.5-Haiku	0.611	0.943	0.742	0.572	0.503	0.535	0.598	0.728	0.657
	Gemini-2.5-Flash	0.714	0.871	0.785	0.468	0.353	0.402	0.618	0.613	0.616
Wiki-NRE	GPT-4o-mini	0.631	0.963	0.762	0.428	0.417	0.422	0.560	0.752	0.642
	GPT-5.1	0.604	0.991	0.751	0.561	0.513	0.536	0.587	0.806	0.679
	Claude-4.5-Haiku	0.578	0.997	0.732	0.536	0.572	0.553	0.562	0.833	0.671
	Gemini-2.5-Flash	0.612	0.978	0.753	0.583	0.621	0.601	0.601	0.840	0.701

Table 6. Anchor discovery statistics and error analysis. $|\overline{\mathcal{A}}|$ and $|\overline{\mathcal{A}}_{\text{gold}}|$ denote the average number of discovered and pseudo-gold anchors per sample, respectively. $\bar{F}_1$ and $\sigma_{\bar{F}_1}$ are the mean and standard deviation of the per-sample Overall F1, respectively. Entity and relation miss rates indicate the fraction of gold anchors of that type not matched by any discovered anchor.

Dataset	Model	$ \overline{\mathcal{A}} $	$ \overline{\mathcal{A}}_{\text{gold}} $	$\bar{F}_1$	$\sigma_{\bar{F}_1}$	Ent. Miss	Rel. Miss
WebNLG	GPT-4o-mini	7.1	7.7	0.698	0.158	17.3%	38.2%
	GPT-5.1	8.5	7.7	0.712	0.149	14.8%	21.7%
	Claude-4.5-Haiku	8.6	7.7	0.727	0.143	13.2%	23.8%
	Gemini-2.5-Flash	8.9	7.7	0.752	0.134	10.8%	20.6%
REBEL	GPT-4o-mini	7.6	8.6	0.618	0.151	10.3%	65.9%
	GPT-5.1	9.8	8.6	0.651	0.141	4.8%	53.2%
	Claude-4.5-Haiku	10.1	8.6	0.668	0.135	4.2%	49.7%
	Gemini-2.5-Flash	7.8	8.6	0.622	0.148	9.8%	64.7%
Wiki-NRE	GPT-4o-mini	7.3	5.8	0.686	0.136	3.7%	58.3%
	GPT-5.1	7.6	5.8	0.700	0.131	0.9%	48.7%
	Claude-4.5-Haiku	8.2	5.8	0.726	0.123	0.3%	42.8%
	Gemini-2.5-Flash	8.0	5.8	0.753	0.115	2.2%	37.9%

4.5.3. Entity vs. Relation Asymmetry

A consistent pattern emerged across all configurations: entity anchors were discovered with substantially higher recall than the relation anchors. On REBEL and Wiki-NRE, entity recall ranged from 0.862 to 0.997, while relation recall spanned only from 0.341 to 0.621. This asymmetry reflects a fundamental difference between the two anchor types; entities correspond to explicit noun phrases that appear verbatim in the source text, making them relatively straightforward for LLMs to identify. Relations, by contrast, are often expressed implicitly through verb phrases, prepositional constructions, or contextual inference, and the mapping from surface forms to canonical relation labels introduces additional ambiguity.

The entity–relation asymmetry is further reflected in the miss rates reported in Table 6. The entity miss rates were uniformly low across the datasets (0.3–17.3%), confirming that anchor discovery rarely failed to identify gold entities. The relation miss rates, however, ranged from 20.6% to 65.9%, with REBEL exhibiting the highest rates due to its large and diverse relation vocabulary (200 types). This finding motivated the design of the schema mapping mechanism φ in Stage 2 and the coverage-aware supplement in Stage 3, both of which compensate for incomplete relation discovery.

4.5.4. Dataset-Dependent Discovery Patterns

The anchor discovery characteristics varied systematically across the datasets in ways that reflect their underlying properties. On WebNLG, the average number of discovered anchors ($|\overline{\mathcal{A}}|$) closely approximated the pseudo-gold count ($|\overline{\mathcal{A}}_{\text{gold}}|$), with ratios ranging from 0.92 to 1.16 across the models. This tight correspondence reflects the clean, structured nature of WebNLG text, where virtually all relational content is explicitly expressed.

On REBEL and Wiki-NRE, the models consistently discovered more anchors than the pseudo-gold count (ratios 0.88–1.41), particularly for GPT-5.1 and Claude 4.5 Haiku. This over-discovery was expected, as the pseudo-gold set was derived from potentially incomplete distant supervision annotations, while the LLMs identified additional valid entities and relations present in the text but absent from the reference. This over-discovery also explains the relatively lower entity precision observed on these datasets (0.578–0.721 vs. pseudo-gold), as many “false positive” entities may in fact correspond to valid but unannotated triplets.

4.5.5. Error Propagation Analysis

To assess whether anchor discovery constitutes a single point of failure for the overall pipeline, we compared anchor discovery's Overall F1 score with the corresponding final KGE Exact F1 from the main results (Tables 2–4) for all 12 model-dataset configurations. Figure 15 visualizes this relationship.

Across all 12 configurations, the final KGE Exact F1 consistently exceeded anchor discovery's Overall F1, with the gap ranging from +0.032 to +0.161 (average +0.087). This demonstrates that anchor discovery is *not* a single point of failure; rather, the multi-stage AEVS pipeline effectively compensates for imperfect anchor coverage. The compensation mechanisms include (1) the schema mapping function φ during grounded extraction, which normalizes relation surface forms to canonical labels independently of whether the exact phrase was discovered as an anchor; (2) the hierarchical restoration matching in Stage 3, which employs fuzzy, schema-based, and text search strategies to verify triplets even when anchor-level matching is incomplete; and (3) the coverage-aware supplement, which targets unused anchors for additional extraction rounds.

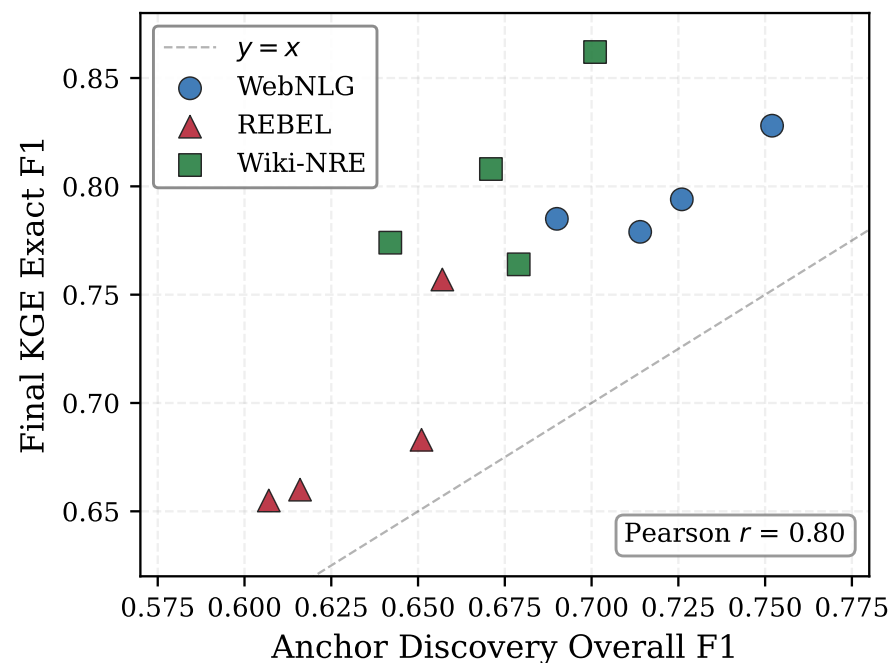


Figure 15. Anchor discovery's Overall F1 vs. final KGE Exact F1 across all 12 model-dataset configurations. The dashed line represents $y = x$. All points lie above this line, indicating that the AEVS pipeline consistently compensated for imperfect anchor discovery through its multi-stage design. Pearson $r = 0.80$.

The positive correlation between the anchor F1 and final KGE F1 (Pearson $r = 0.80$) confirms that anchor quality does meaningfully influence downstream performance, where better anchor discovery leads to better extraction. However, the consistently positive gap demonstrates that the AEVS pipeline adds substantial value beyond what raw anchor quality alone would predict, validating the complementary design of the multi-stage architecture.

4.6. Computational Cost Analysis

A key practical concern for multi-stage LLM pipelines is their computational overhead. To quantify the cost of AEVS, we measured the number of LLM calls and token consumption per sample across all model-dataset configurations. We reconstructed prompts for each pipeline stage using the actual intermediate outputs and estimated the token counts via a

character-based heuristic (character count divided by four, validated to be within $\pm 10\%$ of tiktoken estimates for mixed English and JSON content).

4.6.1. Cost Metrics

Table 7 reports the average number of LLM calls per sample and the average input, output, and total token consumption per sample. For reference, direct prompting requires exactly one LLM call per sample, while EDC requires approximately three calls per sample (extract, define, and canonicalize).

Table 7. Computational cost analysis of AEVS. Avg Calls denotes the mean number of LLM calls per sample. Token counts are per-sample averages (estimated). The theoretical range of LLM calls per sample is from 2 (Stage 1 + Stage 2 only) to 5 (all stages including supplement verification). For comparison, direct prompting requires 1 call/sample, and EDC requires ≈ 3 calls/sample.

Dataset	Model	Avg Calls	Avg Input Tok	Avg Output Tok	Avg Total Tok
WebNLG (1165 samples)	GPT-4o-mini	3.60	12,014	889	12,903
	GPT-5.1	2.83	9694	1002	10,696
	Claude-4.5-Haiku	3.02	10,880	972	11,852
	Gemini-2.5-Flash	3.24	11,347	1718	13,065
REBEL (1000 samples)	GPT-4o-mini	4.28	16,867	1198	18,065
	GPT-5.1	3.58	14,037	1503	15,540
	Claude-4.5-Haiku	4.03	16,592	1439	18,031
	Gemini-2.5-Flash	3.81	15,238	2136	17,374
Wiki-NRE (1000 samples)	GPT-4o-mini	4.14	4618	1166	5784
	GPT-5.1	3.46	3754	1055	4809
	Claude-4.5-Haiku	3.71	4467	994	5461
	Gemini-2.5-Flash	3.36	3856	1648	5504

4.6.2. Analysis

LLM call overhead was moderate. Across all configurations, AEVS required an average of 2.83–4.28 LLM calls per sample. The minimum of two calls corresponds to samples where Stage 1 (anchor discovery) and Stage 2 (grounded extraction) sufficed without triggering any Stage 3 components. The maximum observed averages remained well below the theoretical ceiling of five calls, indicating that the conditional execution design of Stage 3 effectively limits unnecessary computation. Compared with EDC’s approximately three calls per sample, AEVS operated within a similar budget for most model configurations while providing additional provenance guarantees. We observe that WebNLG consistently required the fewest calls across all models (2.83–3.60), reflecting the clean, human-annotated nature of this dataset, where anchor-based extraction produced high-quality triplets that rarely trigger verification or supplement stages. In contrast, the distantly-supervised datasets (REBEL and Wiki-NRE) required more calls (3.36–4.28), as their noisier annotation environments led to more frequent activation of the coverage-aware supplement stage.

Token consumption scaled with schema complexity. The input token counts varied substantially across the datasets, primarily driven by the size of the relation schema included in the Stage 1 and Stage 2 prompts. REBEL, with 200 relation types, incurred the highest input cost (14,037–16,867 tokens/sample), followed by WebNLG with 159 types (9694–12,014 tokens/sample) and Wiki-NRE with 45 types (3754–4618 tokens/sample). The per-call input tokens were remarkably consistent across the models within each dataset (WebNLG = 3337–3603; REBEL = 3921–4117; Wiki-NRE = 1085–1204), confirming that the prompt templates—not model-specific behavior—dominated the input cost. This scaling suggests that for domains with extremely large schemas, schema partitioning or retrieval-based schema selection could further reduce prompt lengths.

Model-specific patterns reflected pipeline dynamics. GPT-4o-mini, the weakest backbone model in our evaluation, consistently required the most LLM calls (3.60–4.28) and incurred the highest total input token overhead, as its lower-quality initial extractions triggered more frequent verification and supplement stages. GPT-5.1, being a stronger model, needed fewer calls (2.83–3.58) and achieved a lower total token consumption despite producing more verbose outputs per call (305–420 output tokens/call vs. 247–282 for GPT-4o-mini). Gemini 2.5 Flash presented a distinctive pattern; it generated the most detailed per-call outputs (490–561 output tokens/call), resulting in the highest output token consumption across all configurations despite requiring a moderate number of calls. This verbose generation behavior contributed to higher total token usage and a longer wall-clock time per sample, representing a practical consideration for deployment.

4.7. Verification Stage Detailed Analysis

To address the reviewers' request for deeper insight into the verification pipeline, we provide a detailed breakdown of how triplets flow through the hierarchical restoration and verification stage (Stage 3). For each initial triplet produced by Stage 2, the restoration module attempts to ground each element (subject, relation, and object) back to the source text or schema. Triplets where all three elements are successfully restored are classified as *fully restored* (FR); partially restored triplets are forwarded to LLM-based verification; and triplets rejected by both restoration and LLM verification are classified as hallucinations. Table 8 reports these statistics.

Figure 16 visualizes the pipeline flow across all configurations, showing how the initial triplets were decomposed into fully restored, partially restored (LLM-approved), and hallucinated components.

Table 8. Verification stage pipeline statistics. Initial = triplets from Stage 2; FR = fully restored (all three elements grounded); Hallu. = hallucinations detected and removed; Supp. = triplets added by coverage-aware supplement; Final = output triplet count. Δ denotes the net change from Initial to Final.

Dataset	Model	Initial	Restoration		Hallucinations		Supp.	Final	$\Delta\%$
			FR	FR%	Detected	Rate%			
WebNLG	GPT-4o-mini	3870	2470	63.8	193	4.99	350	4022	+3.9
	GPT-5.1	4425	3547	80.2	200	4.52	144	4327	−2.2
	Claude-4.5-Haiku	4257	3487	81.9	10	0.23	510	4741	+11.4
	Gemini-2.5-Flash	4356	3820	87.7	43	0.99	55	4368	+0.3
REBEL	GPT-4o-mini	4552	1404	30.8	518	11.38	813	4828	+6.1
	GPT-5.1	6098	3089	50.7	523	8.58	323	5857	−4.0
	Claude-4.5-Haiku	5632	3176	56.4	25	0.44	1016	6616	+17.5
	Gemini-2.5-Flash	5420	2927	54.0	87	1.61	180	5513	+1.7
Wiki-NRE	GPT-4o-mini	4321	1984	45.9	874	20.23	941	4373	+1.2
	GPT-5.1	4296	2682	62.4	632	14.71	411	4032	−6.1
	Claude-4.5-Haiku	3564	2613	73.3	10	0.28	867	4413	+23.8
	Gemini-2.5-Flash	3938	3083	78.3	85	2.16	90	3943	+0.1

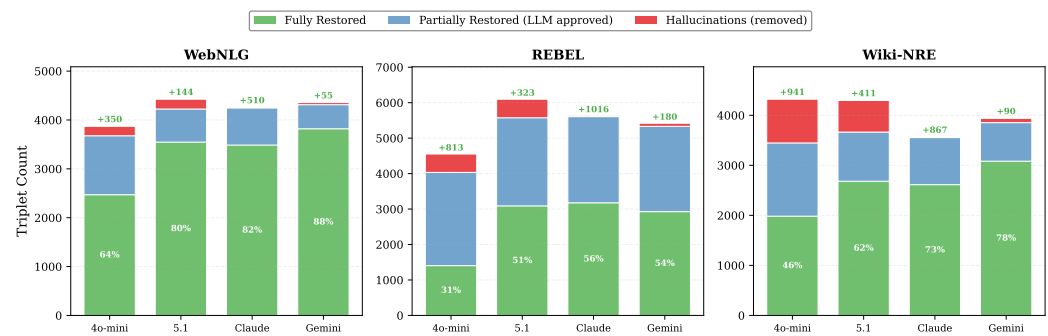


Figure 16. Verification stage pipeline flow across all model-dataset configurations. Each stacked bar decomposed initial triplets into fully restored (green, all three elements grounded), partially restored and LLM-approved (blue), and hallucinations removed (red). Percentages indicate the fully restored rate. Numbers above bars show triplets added by coverage-aware supplement.

4.7.1. Restoration Bottleneck: Relations, Not Entities

A striking finding is that nearly all partial restoration failures (99%+) involved the *relation* element alone, while subjects and objects were almost always restored via exact anchor matching (>99% success rate). Specifically, the dominant failure pattern across all configurations was “S✓ R× O✓”; both entities were successfully grounded in the source text, but the relation could not be mapped to any schema-defined relation type. This validates the anchor-based extraction design. By constraining entities to text-grounded anchors, Stage 1 effectively eliminated entity-level hallucinations. The remaining challenge lies in relation normalization, where the gap between free-form LLM relation expressions and the predefined schema vocabulary creates the primary source of verification failures.

4.7.2. Dataset and Model Patterns

The Fully Restored rate followed a clear hierarchy: WebNLG (63.8–87.7%) > Wiki-NRE (45.9–78.3%) > REBEL (30.8–56.4%). WebNLG’s human-annotated relations aligned more closely with the schema vocabulary, enabling higher restoration rates. The distantly supervised datasets exhibited lower rates due to noisier relation expressions that resisted schema mapping. Across the models, stronger backbones consistently achieved higher FR rates (e.g., Gemini 2.5 Flash reached 87.7% on WebNLG vs. 63.8% for GPT-4o-mini), reflecting their superior ability to generate schema-conformant relation phrases during extraction.

The net triplet change ($\Delta\%$) revealed distinct model behaviors. Claude 4.5 Haiku showed the largest positive Δ (from +11.4% to +23.8%), indicating aggressive supplement-driven coverage expansion with minimal hallucinations. GPT-5.1, conversely, showed a negative Δ (from −2.2% to −6.1%); it generated many initial triplets, but the verification stage removed more than the supplement stage added, reflecting a more conservative pipeline behavior. GPT-4o-mini maintained a modest positive Δ (from +1.2% to +6.1%) because the high hallucination removal rate was partially offset by aggressive supplementation.

4.7.3. Pipeline Arithmetic and Coverage Dynamics

The relationship between these pipeline statistics followed a clear accounting identity: $\text{Final} \approx \text{Initial} - \text{Hallucinations} + \text{Supplement}$. This arithmetic reveals an important insight about the coverage-aware supplement mechanism. For Claude 4.5 Haiku, the supplement stage added substantially more triplets than hallucination removal deleted (e.g., on Wiki-NRE: +867 supplement vs. −10 hallucinations), demonstrating that the coverage monitoring mechanism effectively identified valid triplets missed during the initial extraction. For GPT-5.1, the opposite pattern held; hallucination removal exceeded supplementation (e.g., on Wiki-NRE: −632 hallucinations vs. +411 supplement), indicating that this model generated more speculative triplets that failed verification. These contrast-

ing behaviors suggest that the AEVS pipeline adapts its effective filtering strength to the characteristics of the backbone model, providing stronger correction for more error-prone models while allowing high-quality models to expand coverage.

4.8. Hallucination Rate Analysis

Quantifying hallucination rates is essential for understanding the reliability of LLM-based extraction. We define a *hallucination* as any triplet produced by Stage 2 where neither hierarchical restoration nor LLM-based verification can establish textual grounding. Table 8 reports the hallucination counts and rates. Here, we analyze the type distribution and model-specific patterns.

4.8.1. Overall Hallucination Rates

Hallucination rates varied dramatically across the models, spanning two orders of magnitude. Claude 4.5 Haiku achieved the lowest rates (0.23–0.44%), followed by Gemini 2.5 Flash (0.99–2.16%), GPT-5.1 (4.52–14.71%), and GPT-4o-mini (4.99–20.23%). This ranking was consistent across all three datasets and strongly correlated with the overall model capability. Notably, hallucination rates increased with dataset difficulty—WebNLG (0.23–4.99%) < REBEL (0.44–11.38%) < Wiki-NRE (0.28–20.23%)—reflecting the challenge of extracting well-formed triplets from distantly supervised text. Figure 17 visualizes this two-dimensional pattern, highlighting the interaction between model strength and dataset difficulty.

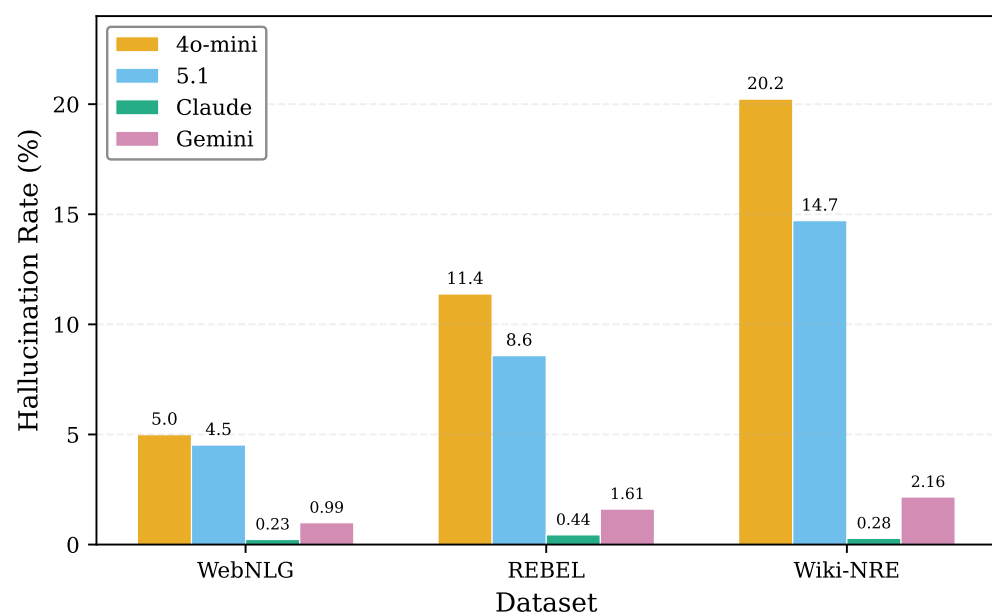


Figure 17. Hallucination rates across models and datasets. Rates span two orders of magnitude from 0.23% (Claude on WebNLG) to 20.23% (GPT-4o-mini on Wiki-NRE). Both model capability and dataset difficulty independently influence hallucination prevalence.

4.8.2. Hallucination Type Distribution

Analysis of the hallucination categories reveals that the dominant type was *Unsupported Relation*, triplets where the subject and object were correctly grounded in the text but the relation between them was not supported by the textual evidence. This type accounted for 95%+ of all detected hallucinations across the configurations. A smaller fraction (1–5%) involved *External Knowledge Injection*, where the model generated triplets using world knowledge not present in the source text (e.g., inferring “country: Italy” for an Italian

film when the nationality was not explicitly stated). Other categories, including *Incorrect Relationship Inference* and *Subject/Object Mismatch*, were rare (<1%).

The overwhelming prevalence of Unsupported Relation hallucinations is consistent with the restoration analysis in Section 4.7. Since anchor-based extraction effectively grounds entities, the primary failure mode is the LLM's tendency to generate plausible but textually unsupported relationship types between correctly identified entities.

4.8.3. Implications for Pipeline Design

These results suggest several directions for future improvements. First, the dominance of Unsupported Relation hallucinations indicates that relation grounding was the primary bottleneck. Potential strategies include (1) providing few-shot relation examples in the extraction prompt to reduce relation vocabulary mismatching, (2) implementing a dedicated relation normalization module between extraction and verification that leverages embedding-based semantic similarity rather than strict string matching, and (3) expanding the schema mapping function φ to support more flexible canonicalization rules.

Second, the strong model-dependence of the hallucination rates has practical implications for deployment. In high-stakes applications where hallucination tolerance is low, the choice of backbone LLM matters substantially; Claude 4.5 Haiku and Gemini 2.5 Flash reduced hallucination rates by 10–50 $\times$ compared with GPT-4o-mini at potentially different computational costs. The verification stage thus serves as a critical safety net, but its effectiveness depends on the baseline hallucination rate of the upstream extraction stage.

Third, the dataset-dependent hallucination patterns suggest that the AEVS pipeline could benefit from adaptive confidence thresholds. On clean datasets like WebNLG, where hallucination rates are already low, a more permissive verification threshold could improve recall without sacrificing precision. On noisier datasets like Wiki-NRE, stricter thresholds are warranted to filter the higher volume of hallucinated content.

4.9. Qualitative Error Analysis

To complement the quantitative analyses, we conducted a qualitative examination of representative failure cases identified by the verification stage.

4.9.1. Error Taxonomy

We classified detected hallucinations into five error types based on which triplet elements failed restoration:

1. **Unsupported Relation ($S\checkmark R\times O\checkmark$):** Both entities are grounded, but the relation lacks textual support. *Example:* From “Los Trancos Creek ... flows ... from Monte Bello Ridge,” the model extracts (Los Trancos Creek, *named after*, Las Trancas), but the text only states the etymological origin and not a “named after” relation.
2. **External Knowledge ($S\checkmark R\times O\times$):** The model injects world knowledge not present in the text. *Example:* From “Addio Alexandra is an Italian romantic drama film, released in 1969,” the model generates (Addio Alexandra, *country*, Italy), inferring the country from “Italian” despite neither the relation “country” nor the entity “Italy” appearing in the text.
3. **Wrong Object ($R\checkmark O\times$):** The relation is valid, but the object is fabricated or incorrectly identified. *Example:* From text about an archery competition, the model generates (archery, *instance of*, sport), where “sport” does not appear in the source text.
4. **Fabricated Content ($S\times R\times$):** Both the subject and relation lack textual grounding, representing more severe hallucinations. These are rare (<1% of errors).
5. **Wrong Subject ($R\checkmark S\times$):** The relation is valid but applied to an incorrect entity. This is also rare in practice.

4.9.2. Error Distribution

Table 9 shows the error type distribution. The unsupported relation type dominated across all configurations, accounting for 97–100% of detected errors. External knowledge injection was the second most common type, primarily affecting GPT-4o-mini on REBEL (10 cases) and Wiki-NRE (5 cases), where the weaker model was more prone to supplementing textual information with parametric knowledge.

Table 9. Error type distribution across datasets and models. Unsup. Rel. = unsupported relation; Ext. Know. = external knowledge injection; Other = remaining types combined.

Dataset	Model	Unsup. Rel.	Ext. Know.	Other	Total
WebNLG	GPT-4o-mini	192	1	0	193
	GPT-5.1	200	0	0	200
	Claude-4.5-Haiku	10	0	0	10
	Gemini-2.5-Flash	41	2	0	43
REBEL	GPT-4o-mini	508	10	0	518
	GPT-5.1	520	2	1	523
	Claude-4.5-Haiku	25	0	0	25
	Gemini-2.5-Flash	84	3	0	87
Wiki-NRE	GPT-4o-mini	869	5	0	874
	GPT-5.1	631	1	0	632
	Claude-4.5-Haiku	10	0	0	10
	Gemini-2.5-Flash	82	3	0	85

4.9.3. Key Insights

The error analysis revealed several important findings with implications for both the current framework and future development.

First, anchor-based extraction was highly effective at preventing entity-level hallucinations; fewer than 1% of the errors involved fabricated subjects or objects. This validates the core design principle of AEVS; by constraining entity mentions to text-grounded anchors, the framework fundamentally limited the model’s ability to introduce non-existent entities. The rare wrong object and wrong subject errors that did occur typically involved near-miss entity references (e.g., confusing “archery” as a sport category vs. an event instance), rather than entirely fabricated entities.

Second, the primary remaining challenge is *relation grounding*, or ensuring that LLM-generated relation phrases correspond to schema-defined types and are supported by the source text. The dominance of unsupported relation errors (97–100%) was consistent across all models and datasets, confirming that this is a systematic limitation of the current pipeline rather than a model-specific weakness. This finding also provides indirect evidence of schema sensitivity. When the relation schema was large and diverse (REBEL with 200 types), the probability of generating a plausible but unsupported relation increased, leading to higher error counts (518–523 for GPT models on REBEL vs. 193–200 on WebNLG).

Third, weaker models (GPT-4o-mini) were significantly more prone to external knowledge injection, suggesting that stronger models better respect the instruction to extract only from the provided text. This model-dependent behavior indicates that prompt adherence—the ability to follow extraction constraints faithfully—is a key differentiator among backbone LLMs for the AEVS framework.

Fourth, the extremely low error counts for Claude 4.5 Haiku (10 errors across all three datasets) and Gemini 2.5 Flash (43–87 errors) demonstrate that state-of-the-art LLMs, when combined with the anchor-based extraction paradigm, can achieve near-zero hallucination rates. This suggests that the AEVS framework effectively amplifies the instruction-following capabilities of strong models while providing a safety net for weaker ones.

4.10. Discussion: Validation of Core Design Principles

The experimental results provide empirical validation for the core insights and design principles proposed in the Introduction and Method sections. We discuss how each key contribution is supported by the experimental evidence.

4.10.1. Anchor-Based Constraint as Anti-Hallucination Mechanism

The Introduction hypothesized that hallucinations in LLM-based extraction arise from the *ungrounded* nature of existing approaches, where models can freely generate triplet elements without textual evidence. The ablation study directly validated this hypothesis; the transition from direct prompting to anchor-based extraction (+AE) yielded the most substantial improvements across all experimental conditions, with gains ranging from +0.117 to +0.324 in Partial F1 across datasets. The consistently larger improvements under the strict and exact criteria (e.g., from +0.267 to +0.389 on Wiki-NRE for strict) confirm that anchor constraints not only improve content accuracy but also enhance structural correctness, precisely because the model can no longer fabricate entity mentions or relation phrases that lack textual grounding.

This finding directly validates our key insight stated in Section 3: *“by establishing a closed vocabulary of text-grounded elements before extraction, we fundamentally limit the model’s ability to fabricate information.”* The experimental evidence demonstrates that this constraint-based approach is more effective than post hoc canonicalization methods (as employed by EDC), achieving improvements from +0.058 to +0.152 over EDC across datasets.

4.10.2. Hierarchical Restoration as Provenance Guarantee

The Method section introduced hierarchical restoration matching with four complementary strategies (exact, fuzzy, schema-based, and text search) to verify triplet grounding. The ablation results for the +RV stage revealed an important insight: the relatively modest F1 changes (from −0.010 to +0.011) indicate that most triplets extracted under anchor constraints were already traceable. This validates the design principle that anchor-based extraction performs the heavy lifting of hallucination elimination, while restoration-based verification provides an explicit *provenance guarantee* rather than serving as a primary filtering mechanism.

The temperature sensitivity analysis further supports this design: Stage 3 (verification and supplement) exhibited variance below 0.020 across all datasets, compared with 0.025–0.071 variance in Stages 1 and 2. This asymmetry confirms that verification involves primarily deterministic matching operations, as designed.

4.10.3. Coverage-Aware Supplement for Comprehensive Extraction

The Introduction identified a tension between precision (avoiding hallucinations) and recall (comprehensive extraction). The coverage-aware supplement mechanism was designed to address this by monitoring anchor utilization and extracting additional triplets for unused anchors. The ablation results confirmed its effectiveness: +SE contributes improvements of up to +0.045 on REBEL and +0.041 on Wiki-NRE, precisely the distantly supervised datasets where annotation incompleteness is most prevalent.

This validates the coverage monitoring strategy described in Section 3.6; by tracking which discovered anchors remain unused after initial extraction, the system can systematically recover valid triplets that would otherwise be missed, particularly for attribute anchors (numerical values and dates) that may be overlooked during initial extraction.

4.10.4. Generalizability Across LLMs and Datasets

A key claim in the Introduction was that AEVS enables extraction “without task-specific training” while adapting to “arbitrary relation types” through prompt engineering. The main results validate this claim; AEVS consistently outperformed specialized trained models (REGEN and GenIE) across all three datasets despite using zero training examples. The framework achieved gains from +0.107 to +0.412 over trained baselines across different relation schema complexities (45–200 types) and annotation qualities (human-annotated WebNLG vs. distantly-supervised REBEL and Wiki-NRE).

Furthermore, the consistent improvements across four different backbone LLMs (GPT-4o-mini, GPT-5.1, Claude 4.5 Haiku, and Gemini 2.5 Flash) demonstrate that the AEVS framework design, rather than any particular model’s capabilities, was responsible for the performance gains. This validates the framework’s generalizability as a model-agnostic approach to grounded knowledge graph extraction.

5. Conclusions

This paper addressed the hallucination problem in LLM-based knowledge graph extraction by proposing anchor-extraction-verification-supplement (AEVS), a framework ensuring complete traceability through character-level provenance linking. AEVS operates in three stages. Anchor discovery establishes a closed vocabulary of text-grounded elements to constrain generation; grounded extraction produces triplets with explicit provenance annotations; and restoration-based verification employs hierarchical matching strategies (exact, fuzzy, schema-based, text search) to validate triplets, complemented by coverage-aware supplement for comprehensive extraction.

Experiments on WebNLG, REBEL, and Wiki-NRE demonstrated that AEVS significantly outperformed the specialized trained models (up to +0.412 F1) and the state-of-the-art LLM-based framework EDC (up to +0.152 F1). Ablation studies revealed that anchor-based extraction served as the primary anti-hallucination mechanism (+0.117 to +0.324 F1), while coverage-aware supplement effectively recovered valid triplets on distantly supervised datasets. Consistent improvements across four backbone LLMs validated AEVS as a model-agnostic approach to grounded knowledge graph extraction.

Our dedicated analyses further characterized the practical behavior of the framework. Anchor discovery achieved F1 scores of 0.607–0.752 across model-dataset configurations, yet the final KGE performance consistently exceeded the anchor-level quality by +0.032–0.161, demonstrating that the multi-stage pipeline compensated for imperfect anchor discovery rather than treating it as a single point of failure. The computational cost remained moderate at 2.83–4.28 LLM calls per sample, comparable to EDC’s approximately three calls per sample while providing additional provenance guarantees. Hallucination rates exhibited strong model dependence, ranging from 0.23% (Claude 4.5 Haiku on WebNLG) to 20.23% (GPT-4o-mini on Wiki-NRE), with the dominant error type being unsupported relations (97–100% of detected hallucinations) rather than entity-level fabrication. This finding confirms that the anchor-based extraction paradigm is highly effective at preventing entity hallucinations, while relation grounding remains the primary challenge.

5.1. Limitations

Several limitations should be noted. First, AEVS currently assumes access to a pre-defined relation schema. In open-domain settings without schema guidance, relation normalization quality degrades, potentially increasing unsupported relation errors. Second, the framework operates at the sentence or passage level without cross-sentence coreference resolution, limiting its ability to extract relations that span multiple sentences. Third, the character-level provenance tracking assumes UTF-8 encoding and has not been evalu-

ated on multilingual or mixed-encoding corpora. Fourth, our evaluation on REBEL and Wiki-NRE was subject to the inherent incompleteness of distant supervision annotations, which may underestimate the true precision of high-recall extraction methods. Finally, while the verification stage provides provenance guarantees, fully restored triplets with coincidental string matches could still represent false positives, though our empirical analysis suggests that this risk is low in practice.

5.2. Future Work

Several directions merit further investigation: (1) *confidence calibration*, or leveraging restoration quality and element-level matching confidence for quantitative uncertainty estimates in high-stakes domains; (2) *task generalization*, or applying the anchor-based paradigm to event extraction, semantic parsing, and slot filling; (3) *document-level extraction*, or extending anchor tracking to support cross-sentence coreference and long-range relations, potentially through integration with existing coreference resolution tools; (4) *RAG integration*, or propagating provenance guarantees through retrieval pipelines for trustworthy large-scale knowledge base construction; (5) *multilingual extension*, or evaluating and adapting the framework for cross-lingual extraction while addressing encoding and tokenization challenges; (6) *adaptive verification thresholds*, or adjusting confidence thresholds based on dataset characteristics and backbone model reliability; and (7) *relation normalization improvements*, or incorporating embedding-based semantic similarity for more flexible relation canonicalization to reduce the dominant unsupported relation error type.

Author Contributions: Conceptualization, Y.Y., Y.Z. and G.C.; methodology, Y.Y., Y.Z. and G.C.; software, Y.Y. and B.H.; validation, Y.Y., Y.Z. and G.C.; formal analysis, Y.Y. and B.H.; investigation, Y.Y., Y.Z. and G.C.; resources, Y.Y., Y.Z. and G.C.; data curation, Y.Y.; writing—original draft preparation, Y.Y. and B.H.; writing—review and editing, G.C. and Y.Z.; visualization, Y.Y. and B.H.; supervision, G.C. and Y.Z.; project administration, G.C.; funding acquisition, Y.Z. All authors have read and agreed to the published version of the manuscript.

Funding: This research was funded by the Ningbo Municipal Project (Grant No. 2024A-154-G) and the Ningbo Key Research and Development Program (Grant No. 2025Z046).

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The datasets used in this study and the source code for AEVS are publicly available at <https://github.com/yyz-nbt/AEVS> (accessed on 8 February 2026).

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Ji, S.; Pan, S.; Cambria, E.; Marttinen, P.; Yu, P.S. A Survey on Knowledge Graphs: Representation, Acquisition, and Applications. *IEEE Trans. Neural Netw. Learn. Syst.* **2022**, *33*, 494–514. [CrossRef] [PubMed]
2. Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Larson, J. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. *arXiv* **2024**, arXiv:2404.16130. [CrossRef]
3. Mintz, M.; Bills, S.; Snow, R.; Jurafsky, D. Distant Supervision for Relation Extraction without Labeled Data. In *Proceedings of the Joint Conference of the 47th Annual Meeting of the ACL and the 4th International Joint Conference on Natural Language Processing of the AFNLP*; Association for Computational Linguistics: Singapore, 2009; pp. 1003–1011.
4. Zheng, S.; Wang, F.; Bao, H.; Hao, Y.; Zhou, P.; Xu, B. Joint Extraction of Entities and Relations Based on a Novel Tagging Scheme. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics*, Vancouver, BC, Canada, 30 July–4 August 2017; pp. 1227–1236.
5. Wei, Z.; Su, J.; Wang, Y.; Tian, Y.; Chang, Y. A Novel Cascade Binary Tagging Framework for Relational Triple Extraction. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Online, 5–10 July 2020; pp. 1476–1488.

6. Wang, Y.; Yu, B.; Zhang, Y.; Liu, T.; Zhu, H.; Sun, L. TPLinker: Single-stage Joint Extraction of Entities and Relations Through Token Pair Linking. In Proceedings of the 28th International Conference on Computational Linguistics, Online, 8–13 December 2020; pp. 1572–1582.
7. Huguet Cabot, P.L.; Navigli, R. REBEL: Relation Extraction By End-to-end Language Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2021*; Association for Computational Linguistics: Punta Cana, Dominican Republic, 2021; pp. 2370–2381.
8. Josifoski, M.; De Cao, N.; Peyrard, M.; Petroni, F.; West, R. GenIE: Generative Information Extraction. In Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Seattle, WA, USA, 10–15 July 2022; pp. 4626–4643.
9. Wei, X.; Cui, X.; Cheng, N.; Wang, X.; Zhang, X.; Huang, S.; Xie, P.; Xu, J.; Chen, Y.; Zhang, M.; Jiang, Y.; Han, W. ChatIE: Zero-Shot Information Extraction via Chatting with ChatGPT. *arXiv* **2023**, arXiv:2302.10205.
10. Wan, Z.; Cheng, F.; Mao, Z.; Liu, Q.; Song, H.; Li, J.; Kurohashi, S. GPT-RE: In-context Learning for Relation Extraction using Large Language Models. In Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing, Singapore, 6–10 December 2023; pp. 3534–3547.
11. Zhang, B.; Soh, H. Extract, Define, Canonicalize: An LLM-based Framework for Knowledge Graph Construction. In Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, Miami, FL, USA, 12–16 November 2024; pp. 9820–9836.
12. Ji, Z.; Lee, N.; Frieske, R.; Yu, T.; Su, D.; Xu, Y.; Ishii, E.; Bang, Y.J.; Madotto, A.; Fung, P. Survey of Hallucination in Natural Language Generation. *ACM Comput. Surv.* **2023**, *55*, 248. [\[CrossRef\]](#)
13. Zelenko, D.; Aone, C.; Richardella, A. Kernel Methods for Relation Extraction. *J. Mach. Learn. Res.* **2003**, *3*, 1083–1106.
14. Zeng, D.; Liu, K.; Lai, S.; Zhou, G.; Zhao, J. Relation Classification via Convolutional Deep Neural Network. In Proceedings of the COLING 2014, the 25th International Conference on Computational Linguistics, Dublin, Ireland, 23–29 August 2014; pp. 2335–2344.
15. Baldini Soares, L.; FitzGerald, N.; Ling, J.; Kwiatkowski, T. Matching the Blanks: Distributional Similarity for Relation Learning. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 28 July–2 August 2019; pp. 2895–2905.
16. Zhong, Z.; Chen, D. A Frustratingly Easy Approach for Entity and Relation Extraction. In Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Online, 6–11 June 2021; pp. 50–61.
17. Dixit, K.; Al-Onaizan, Y. Span-Level Model for Relation Extraction. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 28 July–2 August 2019; pp. 5308–5314.
18. Eberts, M.; Ulges, A. Span-based Joint Entity and Relation Extraction with Transformer Pre-training. In Proceedings of the 24th European Conference on Artificial Intelligence (ECAI 2020), Santiago de Compostela, Spain, 29 August–8 September 2020; pp. 2006–2013.
19. Wadden, D.; Wennberg, U.; Luan, Y.; Hajishirzi, H. Entity, Relation, and Event Extraction with Contextualized Span Representations. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing, Hong Kong, China, 3–7 November 2019; pp. 5784–5789.
20. Wang, J.; Lu, W. Two Are Better Than One: Joint Entity and Relation Extraction with Table-Sequence Encoders. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, Online, 16–20 November 2020; pp. 1706–1721.
21. Shang, Y.M.; Huang, H.; Mao, X.L. OneRel: Joint Entity and Relation Extraction with One Module in One Step. In Proceedings of the AAAI Conference on Artificial Intelligence, Virtual, 22 February–1 March 2022; pp. 11285–11293.
22. Guo, Z.; Zhang, Y.; Lu, W. Attention Guided Graph Convolutional Networks for Relation Extraction. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 28 July–2 August 2019; pp. 241–251.
23. Fu, T.J.; Li, P.H.; Ma, W.Y. GraphRel: Modeling Text as Relational Graphs for Joint Entity and Relation Extraction. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 28 July–2 August 2019; pp. 1409–1418.
24. Zeng, S.; Xu, R.; Chang, B.; Li, L. Double Graph Based Reasoning for Document-level Relation Extraction. In Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, Online, 16–20 November 2020; pp. 1630–1640.
25. Zeng, X.; Zeng, D.; He, S.; Liu, K.; Zhao, J. Extracting Relational Facts by an End-to-End Neural Model with Copy Mechanism. In Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics, Melbourne, Australia, 15–20 July 2018; pp. 506–514.
26. Nayak, T.; Ng, H.T. Effective Modeling of Encoder-Decoder Architecture for Joint Entity and Relation Extraction. In Proceedings of the AAAI Conference on Artificial Intelligence, New York, NY, USA, 7–12 February 2020; pp. 8528–8535.
27. Dognin, P.L.; Padhi, I.; Melnyk, I.; Das, P. ReGen: Reinforcement Learning for Text and Knowledge Base Generation using Pretrained Language Models. In Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing, Online and Punta Cana, Dominican Republic, 7–11 November 2021; pp. 1084–1099.
28. Zhu, Y.; Wang, X.; Chen, J.; Qiao, S.; Ou, Y.; Yao, Y.; Deng, S.; Chen, H.; Zhang, N. LLMs for Knowledge Graph Construction and Reasoning: Recent Capabilities and Future Opportunities. *World Wide Web* **2024**, *27*, 58. [\[CrossRef\]](#)

29. Li, B.; Fang, G.; Yang, Y.; Wang, Q.; Ye, W.; Zhao, W.; Zhang, S. Evaluating ChatGPT's Information Extraction Capabilities: An Assessment of Performance, Explainability, Calibration, and Faithfulness. *arXiv* **2023**, arXiv:2304.11633.
30. Bi, Z.; Chen, J.; Jiang, Y.; Xiong, F.; Guo, W.; Chen, H.; Zhang, N. CodeKGC: Code Language Model for Generative Knowledge Graph Construction. *ACM Trans. Asian Low-Resour. Lang. Inf. Process.* **2024**, *23*, 45. [[CrossRef](#)]
31. Zhang, K.; Gutierrez, B.J.; Su, Y. Aligning Instruction Tasks Unlocks Large Language Models as Zero-Shot Relation Extractors. In *Findings of the Association for Computational Linguistics: ACL 2023*; Association for Computational Linguistics: Stroudsburg, PA, USA, 2023; pp. 794–812.
32. Reimers, N.; Gurevych, I. Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. In Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, Hong Kong, China, 3–7 November 2019; pp. 3982–3992.
33. Ferreira, T.C.; Gardent, C.; Ilinykh, N.; van der Lee, C.; Mille, S.; Moussallem, D.; Shimorina, A. The 2020 Bilingual, Bi-Directional WebNLG+ Shared Task: Overview and Evaluation Results (WebNLG+ 2020). In Proceedings of the 3rd International Workshop on Natural Language Generation from the Semantic Web (WebNLG+), Dublin, Ireland (Virtual), 18 December 2020; pp. 55–76.
34. Distiawan, B.; Weikum, G.; Qi, J.; Zhang, R. Neural Relation Extraction for Knowledge Base Enrichment. In Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics, Florence, Italy, 28 July–2 August 2019; pp. 229–240.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.